

Université de Technologie de Belfort-Montbéliard

DS50 — Project Report

Learning distributions with variational quantum circuits

Ivann VASIC
Ludovic BLONDEAU
Ali HAJERI
Nizar EL ANDALOUSSI BENBRAHIM
Lucas VIGNY

March – June 2026

Contents

Introduction **2**

1 State of the art **3**
1.1 Quantum Circuit Born Machines: general overview . . . 3
1.2 Cost functions for distribution learning . . . 4
1.3 Optimization strategies . . . 5
1.4 Ansatz design . . . 7
1.5 Sources of failure and remedies . . . 9

2 Methodological choices and target distributions **11**
2.1 Target distributions . . . 11
2.2 Cost functions . . . 13
2.3 Optimizers . . . 14
2.4 Ansatz . . . 16
2.5 Evaluation metrics . . . 19
2.6 Implementation choices . . . 20

3 The QCBM Lab tool **22**
3.1 The Single-Run part . . . 22
3.2 The Benchmark part . . . 26
3.3 The Comparison part (Warm-start) . . . 30

4 Experimental results **31**
4.1 Analysis of a single run on the Log-normal distribution (1 to 8 qubits) . . . 31
4.2 Analysis of the results: evolution of QCBM performance . . . 31

5 Warm-start study: initialization from ideal training **33**
5.1 Warm-start advantage on quantum hardware (QPU) . . . 33

6 Execution on a real QPU **35**
6.1 Execution workflow . . . 35
6.2 Automatic backend selection . . . 35
6.3 “Session” mode versus “job” mode . . . 35
6.4 Free quota constraint (10 minutes) . . . 36
6.5 Demonstration: learning a Gaussian on a QPU . . . 36

7 Use case: Quantum Monte-Carlo **40**
7.1 Creation and encoding of the target distribution (single asset) . . . 40
7.2 Trajectory simulation and risk assessment . . . 41
7.3 Perspectives: quantum advantage through multidimensional correlation . . . 42

8 Discussion, limitations and perspectives **43**
8.1 Structural and physical limitations of QCBMs . . . 43
8.2 Perspectives and avenues for improvement . . . 44

9 Conclusion **45**

Introduction

Current quantum computers belong to the NISQ era (*Noisy Intermediate-Scale Quantum*), characterized by a still limited number of qubits and hardware noise that restricts the usable depth of circuits. In this context, **variational quantum algorithms (VQC)** have emerged as the most realistic approach to take advantage of existing hardware: they delegate parameter optimization to a classical processor while only using a short quantum circuit for state preparation. Among their variants, **Quantum Circuit Born Machines (QCBM)** leverage the Born rule to directly turn a quantum state into a probability distribution, which makes them a natural candidate for discrete generative modeling.

Our project follows this line of work: learning target probability distributions with a QCBM, analyzing the practical trade-offs of training (choice of ansatz, cost function, optimizer, measurement regime), then evaluating the robustness of the model against noise, up to execution on a real QPU. Three main questions structure this study: *which ansatz / loss / optimizer combination best suits each type of distribution?*; *how does hardware noise degrade the quality of learning, and which choices make it more robust?*; and *can prior training on an ideal simulator be leveraged to ease learning on a noisy simulator, by reusing the already optimized parameters $\boldsymbol{\theta}$ as a starting point?* The systematic nature of these questions led the team to go beyond the scope of a simple demonstration script and build a genuine interactive experimentation tool.

Contributions.

The contributions of the project fall into four parts, presented here:

- the **complete implementation of a modular QCBM**, covering eight target distributions, seven parameterized ansatzes and all the components required for training (cost functions, gradient-based and gradient-free optimizers, ideal, sampled and noisy evaluation modes);
- the development of an **interactive tool, QCBM Lab**, unifying configuration, training, benchmarking and visualization within a single application, making it possible to run dozens of comparisons without directly touching the code;
- a **systematic comparative study** of configurations (ansatz × loss × optimizer × depth), carried out using the tool's benchmark mode and the automatic hyperparameter search provided by Optuna;
- a **dedicated study of warm-starting** (*warm-start*) ideal → noisy, measuring the quantitative impact of reusing trained parameters on the fidelity achieved in the presence of noise.

Report outline.

The document opens with a **summary of the state of the art**. This is followed by the presentation of the **methodological choices** made for the project, as well as the eight target distributions studied. A chapter is then devoted to the **QCBM Lab tool**: architecture, features and usage. The **experimental results** are then presented in two stages: the detailed analysis of a single run as a function of the number of qubits, then the in-depth study of the warm-start. The following parts report the **execution on a real QPU** as well as a concrete **application use case**. The report concludes with a **discussion** of the limitations and perspectives, followed by the bibliography.

1 State of the art

This chapter reviews the concepts and results needed to understand the rest of the project. It successively covers the general presentation of Quantum Circuit Born Machines, the cost functions used to learn a distribution, the associated optimization strategies, ansatz design, and the main sources of failure encountered in practice. A final subsection summarizes the initial choices that follow from this for the rest of the project.

1.1 Quantum Circuit Born Machines: general overview

1.1.1 From parameterized circuit to variational algorithm

A **parameterized quantum circuit (PQC)** is a circuit composed of fixed quantum gates (used to create entanglement) and gates whose action depends on adjustable continuous parameters, grouped in a vector $\boldsymbol{\theta}$. The PQC is the quantum equivalent of the architecture of a classical neural network.

When a PQC is used within an optimization loop — called a **variational quantum algorithm (VQA)** [1] or *Variational Quantum Circuit* (VQC) — the quantum computer prepares states by applying a series of parameterized gates. The measurements obtained are sent to a classical computer, which updates $\boldsymbol{\theta}$ via an optimization algorithm in order to minimize a cost function.

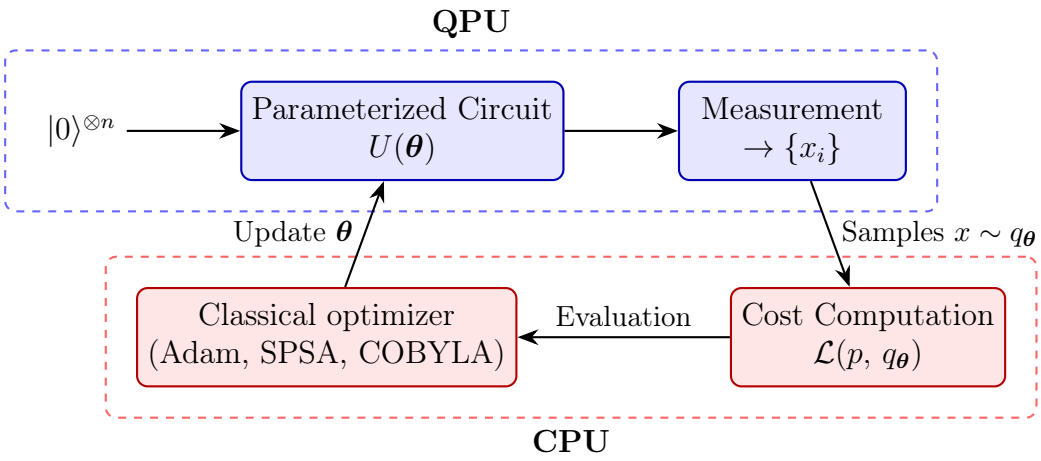


Figure 1: Hybrid loop of a variational quantum algorithm (repeated until convergence).

Advantages of VQAs.

- *NISQ compatibility:* the reduced depth of PQCs limits error accumulation.
- *Versatility:* quantum chemistry, combinatorial optimization, machine learning, generative models.

Challenges.

- *Barren Plateaus:* when scaling up, gradients vanish exponentially.
- *Hardware noise:* decoherence and gate errors distort the cost function.
- *Shot cost:* accurate gradient estimation requires a large number of measurements.

1.1.2 Why quantum for generative modeling?

- **Natural probabilism:** quantum mechanics is intrinsically probabilistic; measurement outcomes directly form a statistical distribution.
- **Expressivity:** VQCs can model complex correlations that are difficult to reproduce with classical neural networks.
- **Potential advantage:** some distributions that are exponentially hard to sample classically would be naturally produced by a quantum circuit.

1.1.3 The QCBM model

A **Quantum Circuit Born Machine (QCBM)** is a specific application of VQCs dedicated to generative modeling. It relies on the **Born rule** to directly turn a quantum amplitude into a probability: the probability of observing the bitstring x upon measurement is

$$q_{\boldsymbol{\theta}}(x) = |\langle x|\psi(\boldsymbol{\theta})\rangle|^2, \qquad |\psi(\boldsymbol{\theta})\rangle = U(\boldsymbol{\theta})|0\rangle^{\otimes n}.$$

Learning pipeline (5 steps).

1. **Initialization (QPU):** n qubits initialized to $|0\rangle$; the workspace is $x \in \{0,1\}^n$ (2^n possible states).
2. **Ansatz (QPU):** application of $U(\boldsymbol{\theta})$ — layers of parameterized rotations and entangling gates (CNOT).
3. **Measurement (QPU):** sampling of $q_{\boldsymbol{\theta}}(x)$ by counting *shots*.
4. **Error computation (CPU):** comparison of $q_{\boldsymbol{\theta}}$ with the target distribution p via a cost function $\mathcal{L}$.
5. **Optimization (CPU):** update of $\boldsymbol{\theta}$ to minimize $\mathcal{L}(p, q_{\boldsymbol{\theta}})$.

1.2 Cost functions for distribution learning

The goal is to minimize

$$\min_{\boldsymbol{\theta}} \ \mathcal{L}(p,\, q_{\boldsymbol{\theta}}).$$

The choice of $\mathcal{L}$ determines convergence speed, noise sensitivity and suitability for gradient-based optimization. Three properties guide this choice for QCBMs.

1.2.1 Desirable properties of a cost function

- **Differentiability:** essential for gradient-based optimizers, which compute $\nabla_{\boldsymbol{\theta}}\mathcal{L}$ via the Parameter-Shift Rule. Gradient-free methods (SPSA, COBYLA) lift this constraint but converge more slowly.
- **Estimation mode:** some losses compare distributions point by point and require estimating all probabilities $q(x)$ — a shot cost exponential in n. Others work from **samples** only (kernels), which makes them naturally suited to execution on a real QPU.
- **Symmetry:** an asymmetric loss penalizes modes missed by $q_{\boldsymbol{\theta}}$ (where p is large and q small) more than a symmetric loss does, which can help or hinder the coverage of all the mass of p.

1.2.2 Information-based measures

Measures from information theory quantify the gap between p and q in terms of surprise or relative entropy. The Kullback–Leibler divergence is the central example:

$$D_{\mathrm{KL}}(p \,\|\, q) = \sum_x p(x) \log \frac{p(x)}{q(x)}.$$

It is zero if and only if $p = q$, and *asymmetric*: it penalizes regions where p is large and q small, without accounting for the excess of q where p is zero. The cross-entropy $H(p, q) = -\sum_x p(x) \log q(x)$ is directly related to it through $H(p, q) = H(p) + D_{\mathrm{KL}}(p\|q)$; minimizing either one over $\boldsymbol{\theta}$ is equivalent since $H(p)$ does not depend on $\boldsymbol{\theta}$.

1.2.3 Direct distances and kernel measures

The quadratic ℓ_2 distance directly compares the probability vectors:

$$\mathcal{L}_2(p, q) = \sum_x \big(p(x) - q(x)\big)^2.$$

It is *symmetric*: it penalizes both under-represented and over-represented states in $q_{\boldsymbol{\theta}}$, which distinguishes it from KL.

Kernel measures (such as the Maximum Mean Discrepancy, MMD) compare two distributions from their samples rather than by accessing the probabilities:

$$\mathrm{MMD}^2 = \mathbb{E}_{x,x' \sim p}[k(x, x')] + \mathbb{E}_{y,y' \sim q}[k(y, y')] - 2\,\mathbb{E}_{x \sim p,\, y \sim q}[k(x, y)],$$

where k is a kernel (Gaussian, for example). The quantum circuit naturally provides samples after measurement, which avoids estimating the 2^n probabilities.

1.2.4 Summary comparison

Family	**Requires exact** $p(x)$	**Differentiable**	**Noise robustness**
Information-theoretic (KL, CE)	Yes	Yes	Medium
Direct distance (ℓ_2)	Yes	Yes	Good
Kernel (MMD)	No (samples)	Yes	Very good

Table 1: Families of cost functions for distribution learning in a QCBM.

On a real QPU, kernel measures are generally preferred as they do not require exhaustive estimation of the 2^n probabilities.

1.3 Optimization strategies

Optimization of the parameters $\boldsymbol{\theta}$ follows the update scheme:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \alpha\, \nabla_{\boldsymbol{\theta}} \mathcal{L},$$

where α is the learning rate. In practice, two main families of optimizers are used.

1.3.1 Gradient-based optimization

Parameter-Shift Rule (PSR)

For a rotation gate $R(\theta) = e^{-i\theta P/2}$ (with P a Pauli matrix), the exact gradient is computed by a discrete shift:

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\mathcal{L}(\theta + \pi/2) - \mathcal{L}(\theta - \pi/2)}{2}.$$

This rule does not amplify measurement noise (unlike classical finite differences) and is therefore suited to execution on a QPU.

Classical gradient optimizers

The most common variants (SGD, RMSprop, Adam, etc.) all follow the scheme $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta\, \hat{g}_t$, where $\hat{g}_t$ is an adapted estimate of the gradient. The differences lie in how $\hat{g}_t$ is computed:

- **Plain SGD:** $\hat{g}_t = \nabla \mathcal{L}$ — non-adaptive step, sensitive to the choice of η.
- **With momentum (SGD+M):** accumulates an average of past gradients to dampen oscillations in erratic directions.
- **Adaptive normalization (RMSprop, Adam):** divides the step by an estimate of the gradient variance, stabilizing convergence in landscapes with heterogeneous curvature. Adam, which combines momentum and normalization, is the recommended gradient optimizer for QCBMs on an ideal simulator.

1.3.2 Gradient-free optimization

These methods do not estimate $\nabla \mathcal{L}$; they are preferable on a QPU because they are robust to measurement noise.

Local linear approximation (e.g. COBYLA)

Evaluates the cost function at several neighboring points, builds a local approximation of $\mathcal{L}$, and chooses the update within a trust region. No gradient required, robust to measurement noise. Slower convergence and possible trapping in a local minimum for large parameter spaces.

Simultaneous perturbation (e.g. SPSA)

Estimates the gradient by *simultaneously* perturbing all d parameters in a random direction $\boldsymbol{\delta} \in \{-1, +1\}^d$: two circuit evaluations suffice regardless of the dimension, which makes this family particularly efficient on a noisy QPU.

1.3.3 Recommended hybrid strategy

1. **Phase 1 — exploration:** use a gradient-free optimizer (e.g. COBYLA) to find a promising region, robust to noise and to local minima.
2. **Phase 2 — refinement:** switch to a gradient-based optimizer (e.g. Adam + PSR on a simulator) or to a simultaneous perturbation method (e.g. SPSA on a QPU) to refine convergence.

1.4 Ansatz design

1.4.1 Definition

Ansatz: parameterized unitary circuit $U(\boldsymbol{\theta})$ structured in L layers:

$$|\psi(\boldsymbol{\theta})\rangle = U(\boldsymbol{\theta})|0\rangle^{\otimes n}, \qquad U(\boldsymbol{\theta}) = \prod_{l=1}^{L}\Big(U_{\text{ent}} \cdot \bigotimes_{j=1}^{n} R_{\alpha_j}(\theta_{l,j})\Big).$$

Each layer alternates parameterized rotations and entangling gates.

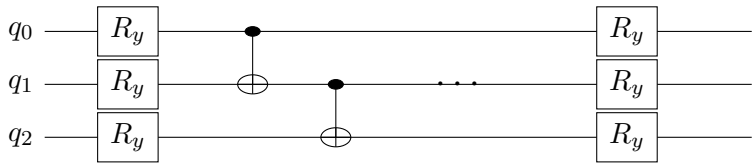
1.4.2 Entanglement topologies

Topology	Layer	Usage	Advantages	Drawbacks
Linear	$n-1$	IBM, Google	Minimal noise	Slow spreading
Circular	n	Cyclic chains	Periodic correlations	Hardware-dependent
Complete	$n(n-1)/2$	Trapped ions	Maximal expressivity	Requires SWAPs
Brick-wall	Variable	General	Good expr./depth ratio	Complex parameterization

Table 2: Entanglement topologies and their usage context.

1.4.3 Illustration: typical structure of a parameterized ansatz

An ansatz frequently used in the NISQ literature is the *Hardware-Efficient Ansatz* (HEA), whose layer structure on n qubits is as follows:



By repeating this block L times:

$$p = 2Ln \text{ parameters}, \qquad N_{\text{CNOT}} = (n-1)L.$$

1.4.4 Performance indicators

Expressivity.

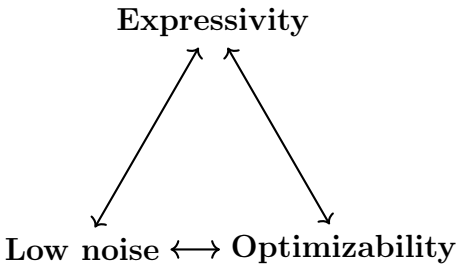
$$\mathcal{E} = D_{\text{KL}}\Big(\hat{P}_{\text{Ansatz}} \,\|\, P_{\text{Haar}}\Big).$$

The smaller $\mathcal{E}$, the more uniformly the ansatz covers the Hilbert space.

Entangling capability.

$$\text{Ent} = \frac{1}{|\Theta|}\int_{\Theta} Q\big(|\psi(\boldsymbol{\theta})\rangle\big)\, d\boldsymbol{\theta}, \qquad Q = \frac{2}{n}\sum_{j=1}^{n}\big(1 - \text{Tr}[\rho_j^2]\big) \in [0,1].$$

1.4.5 The ansatz trilemma



- Expressivity ↑ ⇒ depth ↑ ⇒ noise ↑.
- Noise ↓ ⇒ few gates ⇒ expressivity ↓.
- Optimizability ↑ ⇒ avoid barren plateaus ⇒ limit L.

Rule of thumb: start with $L = 1$ (linear topology) and increase L only if the required accuracy is not reached. Stop if the fidelity drops below 0.85 — then consider error mitigation.

1.5 Sources of failure and remedies

1.5.1 Barren Plateaus

Definition. For a random ansatz on n qubits (approximating a unitary 2-design), the gradient variance decreases exponentially:

$$\mathrm{Var}\left[\frac{\partial \mathcal{L}}{\partial \theta_i}\right] \in O\left(\frac{1}{2^n}\right).$$

On 20 qubits, the variance is $\sim 10^{-6}$: the optimizer is blind.

Consequence on shots. To estimate a gradient with precision ε, the cost is $O(1/\varepsilon^2)$ measurements. Since the gradient standard deviation is $O(2^{-n/2})$, $O(2^n)$ shots would be required — unrealistic in practice. The update turns into a random walk on the landscape of $\mathcal{L}$.

Remedies.

- *Near-identity initialization:* initialize $\boldsymbol{\theta} \approx \mathbf{0}$ so that the circuit initially acts as the identity.
- *Structured ansatz:* use a physics-guided ansatz (UCCSD for chemistry) rather than a deep random HEA.
- *Local cost functions:* evaluate $\mathcal{L}$ qubit by qubit (local cost) rather than on the global state (global cost), which preserves usable gradients.
- *Layer-by-layer training:* progressively add layers and optimize them sequentially.

1.5.2 Hardware noise

Decoherence and gate errors. Qubits interact with their environment: the coherence time T_1 measures energy loss, T_2 phase loss. Each gate typically introduces 0.1–1 % error. A deep circuit sees its state drift towards the maximally mixed state — *noise-induced barren plateau* — which artificially flattens the landscape of $\mathcal{L}$.

Remedies.

- *Short circuits:* stay well below T_1/T_2.
- *Zero-Noise Extrapolation (ZNE):* run the circuit at several amplified noise levels ($\lambda = 1, 2, 3, \ldots$) then extrapolate to $\lambda \to 0$.

1.5.3 The shot problem

A QPU does not return probabilities but discrete states; the circuit must be repeated S times to estimate $q_{\boldsymbol{\theta}}(x)$. With few shots, the estimate fluctuates strongly and the optimizer follows statistical noise rather than a useful gradient.

Recommendation: use between 1000 and 8192 shots to obtain a sufficiently smooth estimate.

1.5.4 Summary table

Source of failure	Symptom	Remedy
Barren Plateau	Gradient ≈ 0, random walk	Init. $\approx$ identity; local cost; structured ansatz
Hardware noise	Slow or oscillating convergence	Short circuits; ZNE; gradient-free optimizers
Few shots	Noisy gradient, instability	$S \in [1000, 8192]$; SPSA on QPU
Local minimum	Premature stagnation	COBYLA in exploratory phase; multi-start
Poor ansatz	Bias too strong or too noisy	Heuristic $L = 1$, increment; trilemma

Table 3: Sources of failure and remedies when training a QCBM.

2 Methodological choices and target distributions

This chapter presents and justifies all the components chosen for the project: the eight target distributions, the five active cost functions, the six optimizers implemented and the seven ansatzes. For each component, the formal definition is accompanied by an analysis of its properties and its relevance in the QCBM context.

2.1 Target distributions

Eight distributions were selected to cover increasing levels of difficulty: sanity-check distributions accessible to the simplest architectures, bimodal distributions testing mode coverage, the canonical QCBM benchmark, a distribution from quantum physics, and a heavy-tailed application distribution.

2.1.1 Basic tests (sanity checks)

Uniform

The uniform distribution assigns the same probability to each of the 2^n computational states:

$$p(x) = \frac{1}{2^n}, \qquad x \in \{0,1\}^n.$$

It is produced exactly by n Hadamard gates; a QCBM should learn it without difficulty, and it serves as a minimal baseline.

Gaussian

Discretization of a centered unimodal Gaussian over the space $\{0, \ldots, 2^n - 1\}$:

$$p(x) \propto \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right), \qquad \mu = \frac{2^n - 1}{2}, \quad \sigma = \frac{2^n}{6}.$$

A model without sufficient expressivity cannot approximate the symmetric bell shape.

Binomial

Exact binomial distribution $\mathcal{B}(2^n - 1,\, 0.5)$:

$$p(k) = \binom{2^n - 1}{k} \left(\tfrac{1}{2}\right)^{2^n - 1}, \qquad k = 0, \ldots, 2^n - 1.$$

Similar to the Gaussian but arising from a discrete process; useful to check that the model generalizes beyond a continuous approximation.

2.1.2 Mode coverage

Bimodal

Superposition of two well-separated Gaussians:

$$p(x) \propto \exp\left(-\frac{(x-\mu_1)^2}{2\sigma^2}\right) + \exp\left(-\frac{(x-\mu_2)^2}{2\sigma^2}\right), \quad \mu_1 = \frac{2^n}{4}, \quad \mu_2 = \frac{3 \cdot 2^n}{4}, \quad \sigma = \frac{2^n}{12}.$$

Classic *mode-collapse* test: a model that is not expressive enough or poorly optimized tends to concentrate on only one of the two modes.

GHZ

Distribution corresponding to the GHZ state (Schrödinger cat state):

$$p(x) = \begin{cases} 0.5 & \text{if } x \in \{\underbrace{00\cdots0}_{n}, \underbrace{11\cdots1}_{n}\}, \\ 0 & \text{otherwise.} \end{cases}$$

Maximal concentration on two diametrically opposed states in the Hilbert space; explicitly requires entanglement (an ansatz without entanglement cannot approximate it).

2.1.3 Canonical QCBM benchmark: Bars and Stripes

The *Bars and Stripes* (BAS) distribution is the reference benchmark for QCBMs [2]. For $n = h \times w$ qubits, each bitstring is interpreted as an $h \times w$ binary image. Only images whose rows are all identical (*stripes*) or whose columns are all identical (*bars*) receive a non-zero probability:

$$p(x) = \begin{cases} \dfrac{1}{N_{\text{valid}}} & \text{if } x \in \text{BAS}(h, w), \\ 0 & \text{otherwise,} \end{cases}$$

where N_{valid} is the number of valid patterns. This distribution is sparse, with structured support and strong correlations between qubits — which makes it a demanding test. The number of qubits must be composite ($4 \to 2 \times 2$, $6 \to 2 \times 3$, $8 \to 2 \times 4$, $9 \to 3 \times 3$, etc.); a prime number gives a trivial factorization $1 \times n$ and BAS degenerates into the uniform distribution.

2.1.4 Quantum physics: Gibbs distribution of a 1D Ising chain

The Gibbs distribution of a ferromagnetic 1D Ising chain with open boundary conditions is:

$$p(\mathbf{s}) \propto \exp(-\beta H(\mathbf{s})), \qquad H(\mathbf{s}) = -J \sum_{i=1}^{n-1} s_i s_{i+1}, \quad s_i \in \{-1, +1\}.$$

With $\beta J = 1$ (moderately ordered regime), the peaks lie on the ferromagnetic configurations $|0\cdots0\rangle$ and $|1\cdots1\rangle$, with a decaying tail over configurations with few domains. Learning thermal states is one of the flagship use cases of QCBMs in the literature.

2.1.5 Application: log-normal distribution

Discretization of the log-normal law on $\{1, \ldots, 2^n - 1\}$:

$$p(k) \propto \frac{1}{k\,\sigma\sqrt{2\pi}} \exp\left(-\frac{(\ln k - \mu)^2}{2\sigma^2}\right), \quad \mu = \ln\frac{2^n}{4}, \quad \sigma = 0.6.$$

Asymmetric with a heavy right tail, characteristic of income distributions, market prices or file sizes. The state $|0\cdots0\rangle$ receives zero probability.

2.1.6 Summary table

Distribution	Family	Key property	Difficulty
Uniform	Sanity-check	Equiprobable, symmetric	Low
Gaussian	Sanity-check	Unimodal, discretized continuous	Low
Binomial	Sanity-check	Unimodal, exact discrete	Low
Bimodal	Mode-coverage	Two separated peaks, mode-collapse	Medium
GHZ	Mode-coverage	Sparse, requires entanglement	High
Bars & Stripes	Benchmark	Structured, correlated, sparse	High
1D Ising Gibbs	Physics	Thermal, local correlations	Medium
Log-normal	Application	Asymmetric, heavy tail	Medium

Table 4: The eight selected target distributions, classified by family and difficulty.

2.2 Cost functions

The project implements five active cost functions. The TVD has a dual status: it can serve both as a cost function for training and as an evaluation metric for the learned distribution; its definition is therefore grouped with the metrics in Section 2.5, and the other four losses are detailed below. For each loss, p denotes the target distribution and $q_{\boldsymbol{\theta}}$ the distribution produced by the QCBM.

2.2.1 Kullback–Leibler divergence

$$\mathcal{L}_{\mathrm{KL}}(p,\, q) = \sum_x p(x) \log \frac{p(x)}{q(x)}.$$

The KL divergence is zero if and only if $p = q$. It is asymmetric: it strongly penalizes regions where p is large and q is small (important states not covered), but ignores regions where q is large and p is zero. It is the reference loss in generative modeling. The relation with cross-entropy is $\mathcal{L}_{\mathrm{CE}} = H(p) + \mathcal{L}_{\mathrm{KL}}(p\|q)$.

2.2.2 Mean squared error (MSE)

$$\mathcal{L}_{\mathrm{MSE}}(p,\, q) = \frac{1}{2^n} \sum_x \big(p(x) - q(x)\big)^2.$$

The MSE symmetrically penalizes deviations, whether they correspond to states under-represented or over-represented by $q_{\boldsymbol{\theta}}$. Its differentiability and symmetry make it robust on an ideal simulator, but it is sensitive to shot noise (statistical fluctuations accumulate squared).

2.2.3 Cross-entropy

$$\mathcal{L}_{\mathrm{CE}}(p,\, q) = -\sum_x p(x) \log q(x).$$

The cross-entropy measures how much $q_{\boldsymbol{\theta}}$ assigns a high probability to the events that matter for p. States with zero probability in p do not contribute to the gradient. Minimizing $\mathcal{L}_{\mathrm{CE}}$ is equivalent to minimizing the KL since $H(p)$ is independent of $\boldsymbol{\theta}$.

2.2.4 Maximum Mean Discrepancy (MMD)

The MMD compares two distributions via a mixture of RBF kernels (bandwidths $\sigma \in \{1, 4, 16\}$):

$$\mathcal{L}^2_{\mathrm{MMD}} = (p-q)^\top K\,(p-q), \qquad K_{ij} = \sum_{\sigma\in\{1,4,16\}} \exp\!\left(-\frac{(i-j)^2}{2\sigma^2}\right).$$

The MMD does not require access to the exact probabilities: the quantum circuit provides samples, and the MMD can be estimated directly from these samples. It is differentiable, naturally suited to a shot-based setting, and robust to measurement noise — which makes it particularly appropriate on a QPU.

2.2.5 Comparison table

Loss	**Symmetric**	**Requires exact p**	**Differentiable**	**Noise robustness**
KL Divergence	No	Yes	Yes	Medium
MSE	Yes	Yes	Yes	Good
Cross Entropy	No	Yes	Yes	Medium
MMD	Yes	No	Yes	Very good

Table 5: Comparison of the other four cost functions (the TVD, with its dual loss/metric use, is covered in Section 2.5).

2.3 Optimizers

Six optimizers are implemented, organized into two families depending on whether or not they require computing the gradient.

2.3.1 Gradient-based optimizers

These four optimizers use the *Parameter-Shift Rule* (PSR) to compute the exact gradients $\partial\mathcal{L}/\partial\theta_i$ via two circuit evaluations per parameter, or PennyLane's analytic autodifferentiation.

SGD

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta\,\nabla_{\boldsymbol{\theta}}\mathcal{L}.$$

Basic update without memory. Simple to diagnose but sensitive to the choice of the learning rate η.

SGD + Momentum

$$v_t = \gamma v_{t-1} + \nabla_{\boldsymbol{\theta}} \mathcal{L}, \qquad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta v_t.$$

The momentum term $\gamma \approx 0.9$ dampens oscillations and accelerates convergence along consistent directions of the energy landscape.

RMSprop

$$E[g^2]_t = \gamma E[g^2]_{t-1} + (1-\gamma)(\nabla \mathcal{L})^2, \qquad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \frac{\eta}{\sqrt{E[g^2]_t + \varepsilon}} \nabla \mathcal{L}.$$

Normalizes the step by the gradient history; suited to landscapes with heterogeneous curvature across parameters.

Adam

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla \mathcal{L}, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla \mathcal{L})^2, \quad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon},$$

with $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$. Adam combines momentum and adaptive normalization; it is the most recommended gradient-based optimizer on an ideal simulator for QCBMs.

2.3.2 Gradient-free optimizers

These two optimizers do not estimate $\nabla \mathcal{L}$; they are robust to measurement noise and can be used on a real QPU without parameter-shift overhead.

SPSA

Simultaneous Perturbation Stochastic Approximation. At each iteration, the d parameters are perturbed simultaneously by a random vector $\boldsymbol{\delta} \in \{-1, +1\}^d$ (Rademacher distribution):

$$\hat{g}_t = \frac{\mathcal{L}(\boldsymbol{\theta} + c\boldsymbol{\delta}) - \mathcal{L}(\boldsymbol{\theta} - c\boldsymbol{\delta})}{2c} \, \boldsymbol{\delta}^{-1}, \qquad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \, \hat{g}_t.$$

Two circuit evaluations suffice regardless of the dimension d, which makes SPSA particularly efficient on a QPU.

COBYLA

Constrained Optimization BY Linear Approximation. At each iteration, several points $\boldsymbol{\theta} + \delta_i$ are evaluated to build a local linear approximation of $\mathcal{L}$, then an update is chosen within the trust region. The implementation relies on `scipy.optimize.minimize(..., method="COBYLA")` with a maximum number of internal iterations limited to 5 per call, which gives the optimizer a chance to explore without monopolizing the circuit budget.

2.3.3 Summary table

Optimizer	Family	Recommended for	Circuits/iter
SGD	Gradient	Diagnostics, baseline	$2d$
SGD + Momentum	Gradient	Oscillating landscapes	$2d$
RMSprop	Gradient	Heterogeneous gradients	$2d$
Adam	Gradient	Ideal simulator	$2d$
SPSA	Gradient-free	Real QPU, noisy simulator	2
COBYLA	Gradient-free	Exploration, local minima	$O(d)$

Table 6: The six implemented optimizers. d = number of circuit parameters.

2.4 Ansatz

Seven ansatzes are available in QCBM Lab. They span a spectrum ranging from architectures without entanglement up to all-to-all topologies, in order to measure the impact of connectivity and expressivity on learning quality. The circuits below illustrate each ansatz for $n = 3$ qubits and $L = 1$ layer (except BrickWall: $n = 4$, $L = 2$ to visualize the alternating pattern).

2.4.1 BasicEntangler

Parameterized R_X rotations on each qubit, followed by a cycle of CNOTs in a linear chain. Minimal architecture with entanglement.

$$N_{\text{params}} = L \cdot n, \qquad \text{shape} = (L,\, n).$$

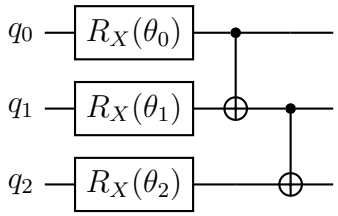


Figure 2: BasicEntangler — $n = 3$, $L = 1$.

2.4.2 StronglyEntangling

R_Z–R_Y–R_Z rotations on each qubit (general U_3 decomposition) followed by CNOTs with ranges varying by layer. Higher expressivity than BasicEntangler for the same number of layers.

$$N_{\text{params}} = 3Ln, \qquad \text{shape} = (L,\, n,\, 3).$$

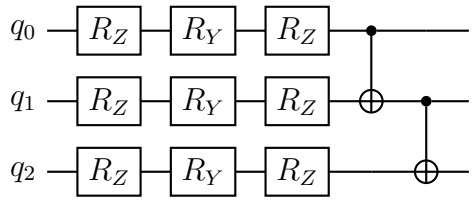


Figure 3: StronglyEntangling — $n = 3$, $L = 1$.

2.4.3 HardwareEfficient

R_Y–R_Z rotations per qubit and CNOT entanglement in a linear chain. Native structure for IBM processors with linear connectivity.

$$N_{\text{params}} = 2Ln, \qquad N_{\text{CNOT}} = (n-1)L, \qquad \text{shape} = (L,\, n,\, 2).$$

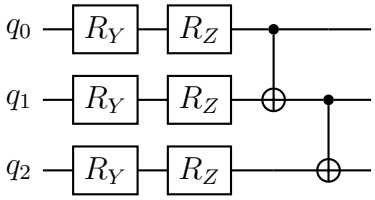


Figure 4: HardwareEfficient — $n = 3$, $L = 1$.

2.4.4 RotationOnly

Ansatz *without entanglement*: only R_Y rotations per qubit. Baseline to measure the contribution of entanglement. Can only represent product distributions $p(x) = \prod_i p_i(x_i)$ — unable to learn GHZ or BAS.

$$N_{\text{params}} = Ln, \qquad N_{\text{CNOT}} = 0, \qquad \text{shape} = (L,\, n).$$

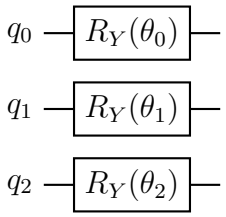


Figure 5: RotationOnly — $n = 3$, $L = 1$. No entangling gate.

2.4.5 AllToAll

R_X–R_Y–R_Z rotations on each qubit, then CNOTs between all pairs of qubits. Maximal expressivity but $O(n^2)$ depth per layer — suited to small circuits, costly on real hardware.

$$N_{\text{params}} = 3Ln, \qquad N_{\text{CNOT}} = \frac{n(n-1)}{2} \cdot L, \qquad \text{shape} = (L,\, n,\, 3).$$

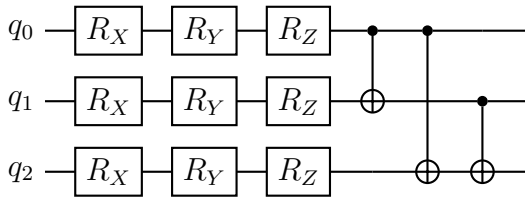


Figure 6: AllToAll — $n = 3$, $L = 1$. CNOT between all pairs (q_0, q_1), (q_0, q_2), (q_1, q_2).

2.4.6 BrickWall

Alternating pattern ("brick wall"): even layers connect the pairs $(0, 1), (2, 3), \ldots$ and odd layers connect $(1, 2), (3, 4), \ldots$ Constant depth in $O(L)$, good expressivity/depth ratio. Inspired by tensor-network circuits (MPS/PEPS).

$$N_{\text{params}} = 2Ln, \qquad \text{shape} = (L,\, n,\, 2).$$

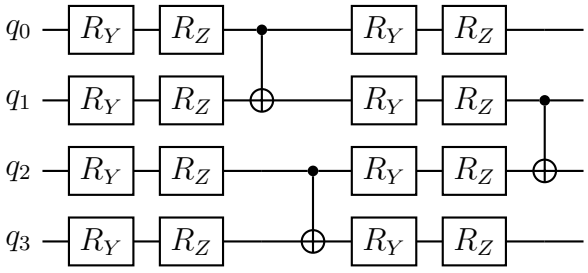


Figure 7: BrickWall — $n = 4$, $L = 2$. Layer 0 (even): CNOT$(0, 1)$ and CNOT$(2, 3)$; layer 1 (odd): CNOT$(1, 2)$.

2.4.7 DataReuploading

Inspired by Pérez-Salinas et al. (2020): each layer applies three rotations R_Z–R_Y–R_Z (U_3 equivalent) then a circular ring of CNOTs — qubit $n - 1$ loops back to qubit 0. Favors periodic correlations.

$$N_{\text{params}} = 3Ln, \qquad N_{\text{CNOT}} = n \cdot L, \qquad \text{shape} = (L,\, n,\, 3).$$

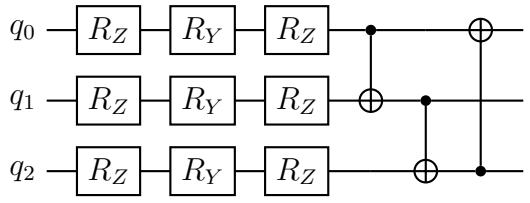


Figure 8: DataReuploading — $n = 3$, $L = 1$. Circular ring: CNOT$(0, 1)$, CNOT$(1, 2)$, CNOT$(2, 0)$.

2.4.8 Comparison table

Ansatz	Rotations	Entanglement	N_{params}	Depth
BasicEntangler	R_X	Linear chain	Ln	$O(L)$
StronglyEntangling	$R_Z R_Y R_Z$	Chain (var. range)	$3Ln$	$O(L)$
HardwareEfficient	$R_Y R_Z$	Linear chain	$2Ln$	$O(L)$
RotationOnly	R_Y	None	Ln	$O(L)$
AllToAll	$R_X R_Y R_Z$	All-to-all	$3Ln$	$O(Ln^2)$
BrickWall	$R_Y R_Z$	Alternating	$2Ln$	$O(L)$
DataReuploading	$R_Z R_Y R_Z$	Circular ring	$3Ln$	$O(L)$

Table 7: Comparison of the seven ansatzes implemented in QCBM Lab.

2.5 Evaluation metrics

To quantify the similarity between a target distribution P and a distribution Q generated by our QCBM, we rely on two reference metrics: the total variation distance (TVD) and the fidelity.

Total Variation Distance (TVD)

The total variation distance (*Total Variation Distance*) is a statistical distance measure. It represents the largest difference in probability that the two distributions can assign to the same state. Visually, it corresponds to the non-overlapping area between the two probability distributions.

Mathematically, for two discrete distributions P and Q defined on a state space X, the TVD is defined by:

$$\text{TVD}(P, Q) = \frac{1}{2} \sum_{x \in X} |P(x) - Q(x)| \tag{1}$$

- **Interpretation:** The TVD lies between 0 and 1. A value of 0 indicates that the distributions are perfectly identical, while a value of 1 means that they are completely disjoint.
- **Dual use:** In our project, the TVD has a dual status: it is both one of the five available cost functions that the optimizer can seek to minimize (towards 0) and an evaluation metric for the quality of the learned distribution.

Fidelity

Unlike the TVD, which is an error measure, the fidelity is a similarity measure. Commonly used in quantum mechanics and information theory, it quantifies the overlap or geometric affinity between two distributions.

The classical fidelity between P and Q is computed by summing the square roots of the products of their respective probabilities for each state:

$$F(P, Q) = \sum_{x \in X} \sqrt{P(x) \cdot Q(x)} \tag{2}$$

- **Interpretation:** The fidelity also ranges from 0 to 1. A value of 1 indicates a perfect overlap (the distributions are identical), while a value of 0 indicates a complete absence of overlap.
- **Sensitivity:** The presence of the square root makes this metric particularly sensitive to zero probabilities. If the model generates a probability of 0 for a state expected by the target distribution (GHZ distribution), the product is zero, which strongly penalizes the overall score. The goal is to maximize this metric (towards 1) during training.

2.6 Implementation choices

Before starting the implementation, we decided to adopt an architecture separating the frontend from the backend. This approach promotes the modularity and maintainability of the project. It also offers greater flexibility by making it possible, in the future, to replace or evolve the user interface (for example towards an HTML interface) without impacting the backend features.

2.6.1 Frontend

For the frontend, we chose Tkinter because of its status as the standard reference for desktop GUI development in Python. As a built-in library, it benefits from a vast ecosystem with rich documentation, a large community and countless available resources. Moreover, its intuitive syntax and ease of implementation allowed us to quickly develop a clean and functional interface, without the heaviness or steep learning curve associated with more complex external frameworks.
Furthermore, Tkinter integrates seamlessly with Matplotlib, which makes incorporating dynamic charts and other advanced data visualizations directly into the application remarkably easy.

2.6.2 Backend

For the backend, we used Docker to containerize our application and ensure a smooth development and deployment workflow. Docker is an industry standard for virtualization, allowing us to package the backend environment, its dependencies and its libraries into a single isolated container. This approach eliminates the classic "it works on my machine" problem by ensuring that the backend runs identically on any operating system. Its lightweight nature, ease of configuration via Dockerfiles, and excellent scalability considerably simplified the management of our architecture and accelerated the integration process.
This ensures that the entire virtual hardware environment is fully reproducible: anyone who clones the repository can easily launch and run QCBM Lab with a single command, without worrying about local configuration conflicts.

2.6.3 Optuna

In order to best configure and fine-tune the hyperparameters of our optimizers and ansatzes (such as the learning rate η, the number of layers, etc.), we chose to integrate Optuna. Now a true industry standard for hyperparameter search (*Hyperparameter Optimization - HPO*), Optuna provides an automatic and intelligent exploration system, designed to efficiently navigate large search spaces. Its use proves essential to handle the complexity of variational quantum algorithms: it makes it possible to quickly isolate the best combinations where a

manual search would be impossible, considerably increasing the robustness and quality of our convergence results.

3 The QCBM Lab tool

3.1 The Single-Run part

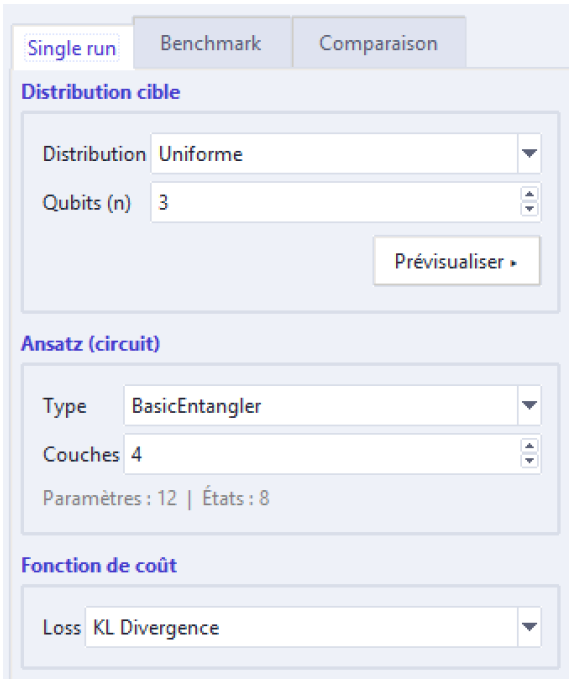
During this project, we chose to develop and use a graphical interface in order to test different configurations as simply as possible. It is not a mere gadget, since the interface meets a real experimental need. The interface was built with Tkinter, while the backend runs independently via Docker.

On launch, the application offers three main tabs: *Single run*, *Benchmark*, *Comparison*, *Monte-Carlo*.

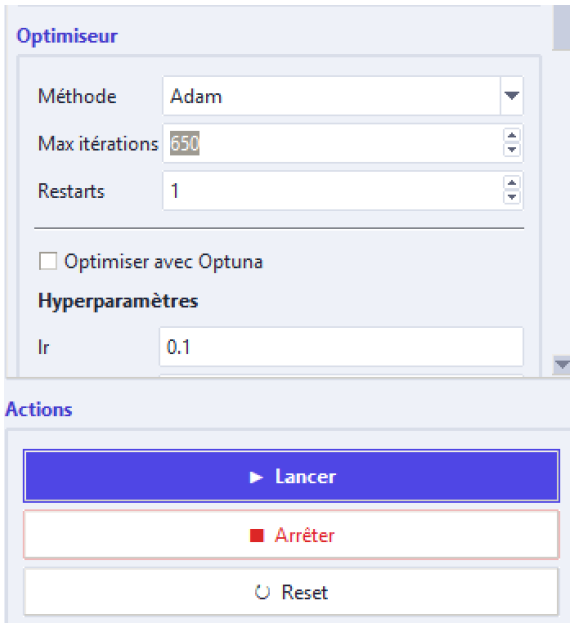
The Single Run tab

This is the main tab, used to run a test on a given configuration. The user can select:

- A target distribution (Uniform, Gaussian, Log-normal, etc.) and the number of qubits (n).
- A parameterized ansatz type (BasicEntangler, HardwareEfficient, etc.) as well as its number of layers (L).
- A cost function to minimize (KL Divergence, MSE, TVD, etc.).
- An optimizer (Adam, SPSA, COBYLA, etc.) with fixed hyperparameters (learning rate η, maximum number of iterations).
- The option to enable hyperparameter optimization via Optuna.



(a) Parameter configuration



(b) Optimizer configuration and actions

Figure 9: Configuration interface of the Single run tab.

On the right side of the screen, live results are displayed as detailed metrics and charts: evolution of the loss function (*loss*), fidelity, TVD, as well as various tabs for exploring the model.

Demonstration of a training run (Single Run)

To concretely illustrate the use of this tab, we configured a learning task on a **Log-normal** distribution with **5 qubits**, using the **BasicEntangler** ansatz with **4 layers**, the **KL Divergence** as cost function and the **Adam** optimizer.

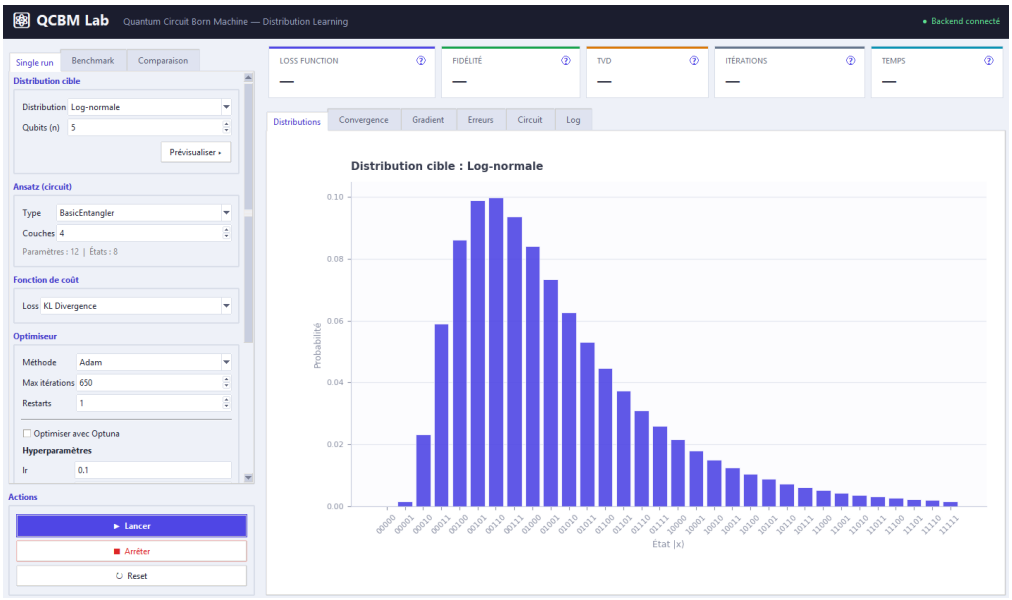


Figure 10: Preview of the target distribution (Log-normal on 5 qubits) via the preview button.

At the end of the training iterations, the QCBM Lab tool generates several comprehensive visualizations to analyze the results directly within the interface:

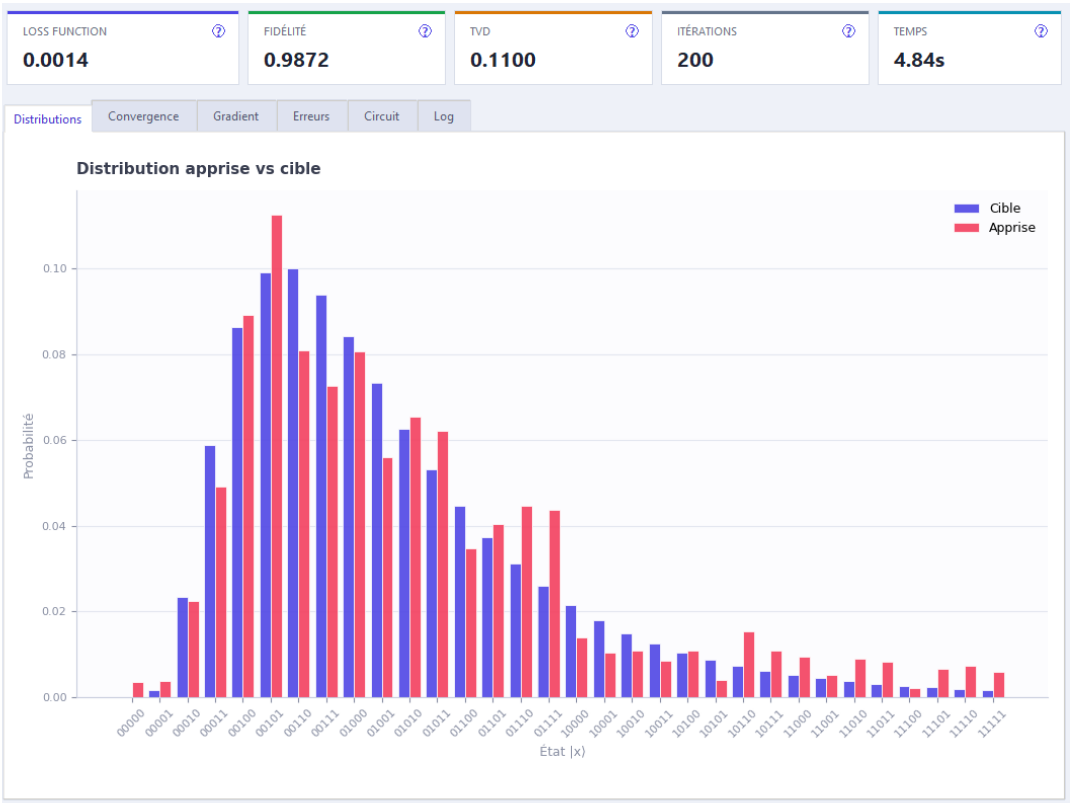


Figure 11: Comparison of state probabilities: target (blue) and learned ansatz (red).

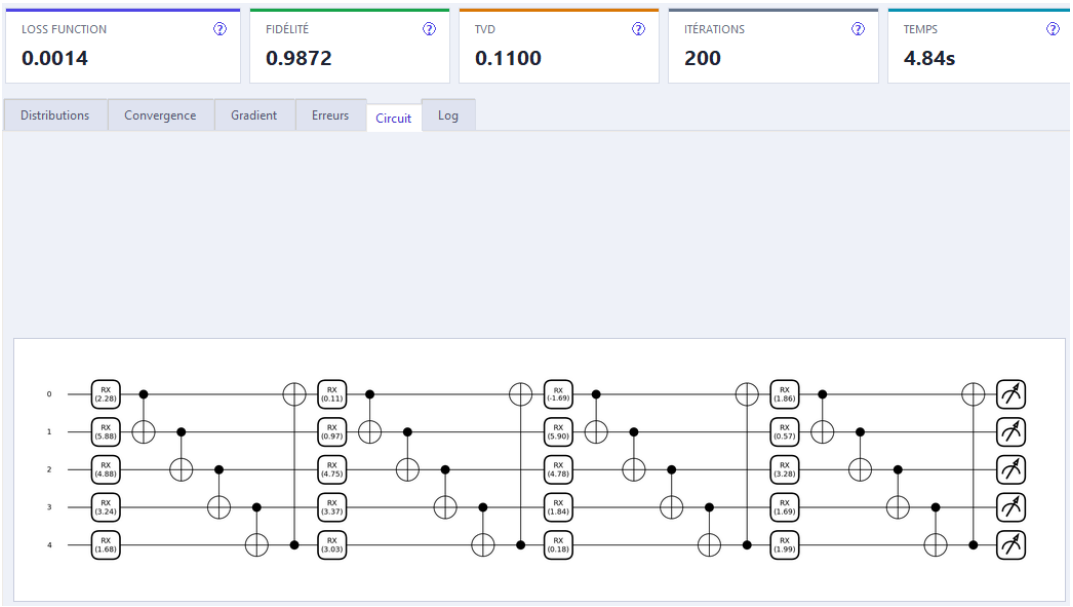


Figure 12: View of the optimized parameterized ansatz with its final rotation angles.



Figure 13: Convergence curve of the KL Divergence with interactive history.



Figure 14: Logarithmic tracking of the gradient norm to detect potential plateaus.



Figure 15: Relative probability difference (error) for each bitstring.

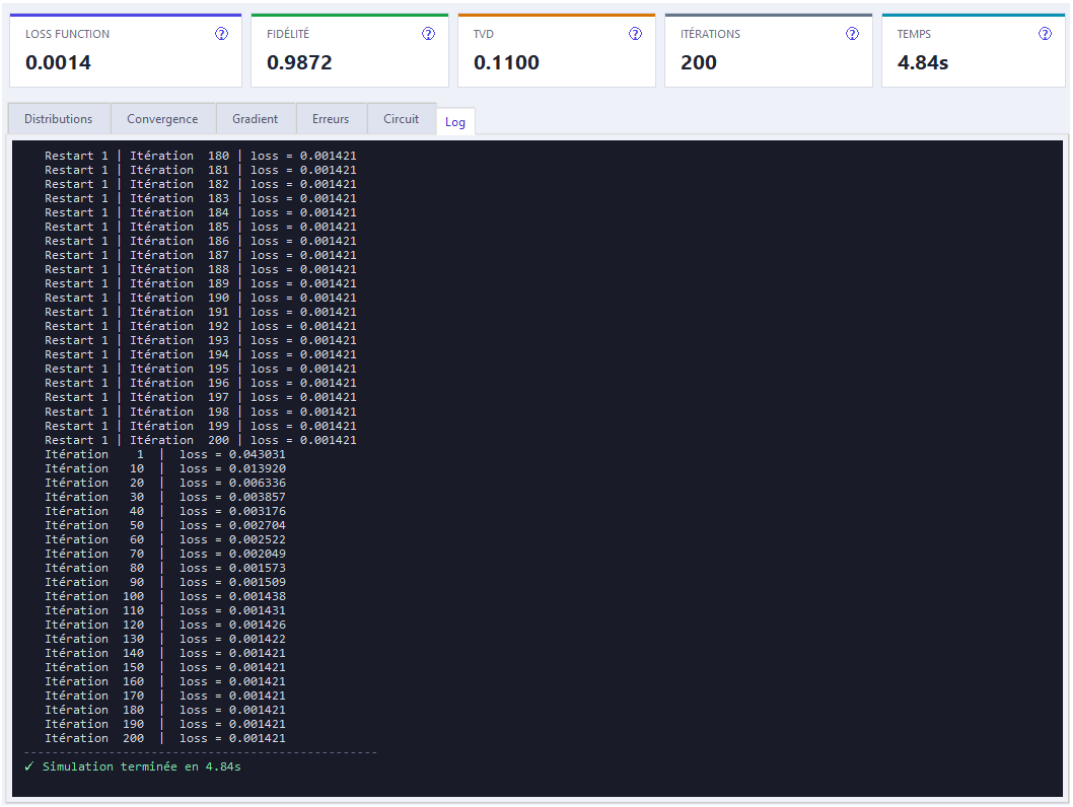


Figure 16: Execution logs showing the step-by-step progress of training.

3.2 The Benchmark part

The second tab of our interface is dedicated to *Benchmark*. This section makes it possible to determine the best possible configuration for a given distribution by automating the trials.

As illustrated in the figure below, it is possible to launch a complete comparative test. To avoid excessive computation times, the user can finely select the architectures to evaluate: the different ansatzes, the depth ranges, the cost functions and the optimizers. They can also set the budget of quantum samples (*shots*) or simulate a complete noisy environment. Optuna can optionally be integrated to automate the discovery of optimal hyperparameters (such as searching for the best *lr* for each sub-configuration).

Once the benchmark is complete, a global PDF export of all configurations is offered in order to compare, archive or communicate the results.

To illustrate this feature, we will study a specific case. We want to determine which configuration is the most effective for simulating a Log-normal distribution on 3 qubits. We therefore select the BasicEntangler, HardwareEfficient and RotationOnly ansatzes, the KL and MMD cost functions, the Adam (gradient-based) and SPSA (gradient-free) optimizers, as well as a range of 1 to 2 layers for the ansatz. This gives a total of 24 combinations, which we will run on the noisy simulator.

Here is the first page of the document exported after running the benchmark, containing information on the target distribution as well as the details of the best-performing configuration.

We observe that the ideal configuration is a HardwareEfficient ansatz with 2 layers, with a KL cost function and an SPSA optimizer, the latter having as hyperparameters a learning rate

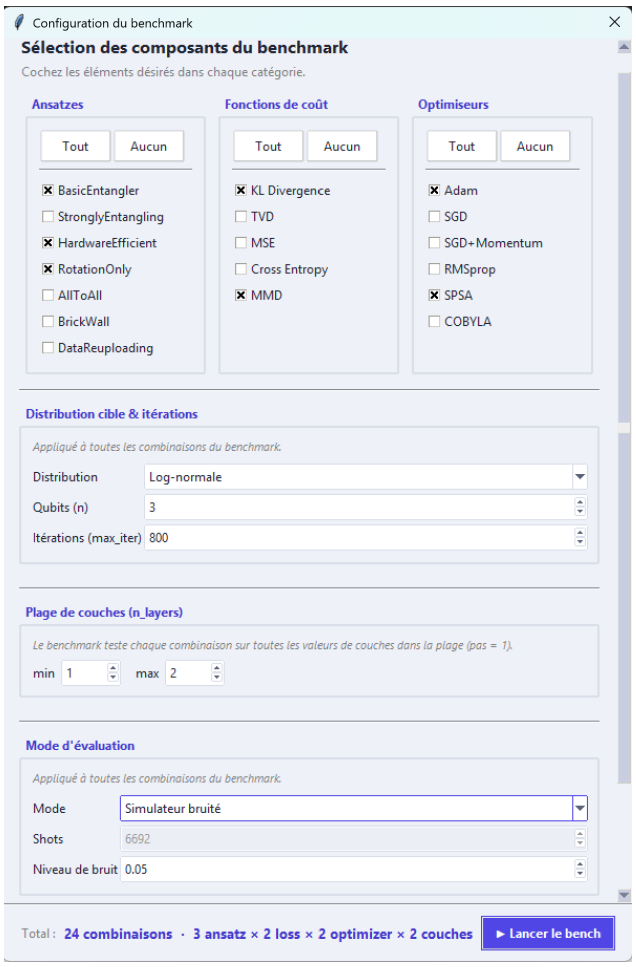
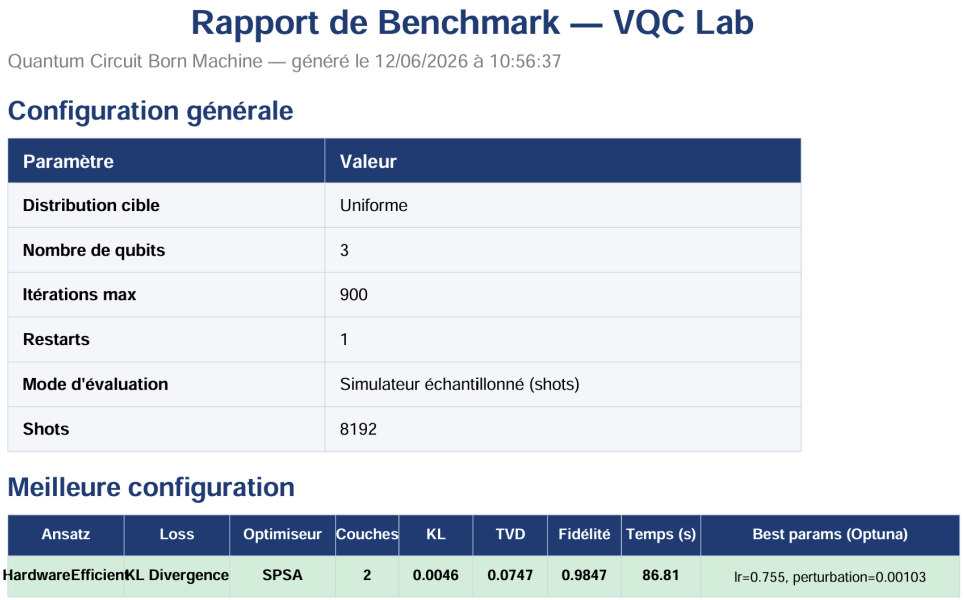


Figure 17: Combinatorial configuration window for a complete benchmark.



Rapport de Benchmark — VQC Lab

Quantum Circuit Born Machine — généré le 12/06/2026 à 10:56:37

Configuration générale

Paramètre	Valeur
Distribution cible	Uniforme
Nombre de qubits	3
Itérations max	900
Restarts	1
Mode d'évaluation	Simulateur échantillonné (shots)
Shots	8192

Meilleure configuration

Ansatz	Loss	Optimiseur	Couches	KL	TVD	Fidélité	Temps (s)	Best params (Optuna)
HardwareEfficient	KL Divergence	SPSA	2	0.0046	0.0747	0.9847	86.81	lr=0.755, perturbation=0.00103

Figure 18: Cover page of the benchmark report.

of 0.755 and a perturbation of 0.00103. The PDF export also contains detailed information on the training of the best-performing configuration (cost function curves, final simulation and metrics). In addition, the detailed list of the best-performing configurations (sorted by decreasing fidelity) is available (see the image below).

Résultats détaillés (24 configurations testées, triées par fidélité décroissante)

#	Ansatz	Loss	Optimiseur	Couches	KL	TVD	Fidélité	Temps (s)	Best params (Optuna)
1	**HardwareEfficient**	**KL Divergence**	**SPSA**	**2**	**0.0046**	**0.0747**	**0.9847**	**86.81**	lr=0.755, perturbation=0.00103
2	HardwareEfficient	MMD	Adam	2	0.0012	0.0757	0.9826	52.89	lr=0.551, beta1=0.856, beta2=0.933, eps=2.2e-10, weight_decay=0.000373
3	HardwareEfficient	MMD	SPSA	2	0.0010	0.0783	0.9820	85.84	lr=0.755, perturbation=0.00103
4	HardwareEfficient	KL Divergence	Adam	2	0.0079	0.1123	0.9802	51.18	lr=0.955, beta1=0.842, beta2=0.936, eps=1.61e-10, weight_decay=0.00164
5	BasicEntangler	KL Divergence	SPSA	2	0.0163	0.1760	0.9495	77.53	lr=0.755, perturbation=0.00103
6	BasicEntangler	MMD	Adam	2	0.0078	0.1946	0.9464	45.50	lr=0.551, beta1=0.856, beta2=0.933, eps=2.2e-10, weight_decay=0.000373
7	BasicEntangler	KL Divergence	Adam	2	0.0169	0.1545	0.9459	45.03	lr=0.551, beta1=0.856, beta2=0.933, eps=2.2e-10, weight_decay=0.000373
8	BasicEntangler	MMD	SPSA	2	0.0070	0.2020	0.9448	80.15	lr=0.97, perturbation=0.00827
9	BasicEntangler	KL Divergence	SPSA	1	0.0287	0.1663	0.9431	50.06	lr=0.895, perturbation=0.00104
10	BasicEntangler	KL Divergence	Adam	1	0.0287	0.1669	0.9429	34.14	lr=0.551, beta1=0.856, beta2=0.933, eps=2.2e-10, weight_decay=0.000373
11	HardwareEfficient	MMD	Adam	1	0.0101	0.2165	0.9334	30.71	lr=0.932, beta1=0.866, beta2=0.929, eps=1.36e-10, weight_decay=0.00164
12	HardwareEfficient	MMD	SPSA	1	0.0099	0.2124	0.9329	52.65	lr=0.92, perturbation=0.00127
13	HardwareEfficient	KL Divergence	SPSA	1	0.0230	0.1992	0.9321	52.80	lr=0.351, perturbation=0.0148
14	HardwareEfficient	KL Divergence	Adam	1	0.0230	0.2023	0.9321	29.42	lr=0.551, beta1=0.856, beta2=0.933, eps=2.2e-10, weight_decay=0.000373
15	BasicEntangler	MMD	Adam	1	0.0580	0.3805	0.8860	28.86	lr=0.028, beta1=0.568, beta2=0.929, eps=2.92e-09, weight_decay=0.0456
16	RotationOnly	KL Divergence	Adam	1	0.0381	0.3084	0.8834	19.66	lr=0.551, beta1=0.856, beta2=0.933, eps=2.2e-10, weight_decay=0.000373
17	RotationOnly	KL Divergence	SPSA	1	0.0380	0.3035	0.8822	32.72	lr=0.755, perturbation=0.00103
18	RotationOnly	KL Divergence	SPSA	2	0.0380	0.3035	0.8822	44.26	lr=0.755, perturbation=0.00103

Figure 19: List of the best-performing configurations of the benchmark.

The HardwareEfficient ansatz largely dominates the ranking, taking the top four places with fidelity scores above 0.98. Conversely, the RotationOnly ansatz systematically occupies the last places (ranks 16 to 18), with fidelity plateauing around 0.88. This confirms in practice that an architecture without entangling gates is unable to capture the correlations of a complex asymmetric distribution such as the Log-normal. BasicEntangler, for its part, offers a middle-ground compromise, ranking between 5 and 10 with fidelities around 0.94.

Going from 1 to 2 layers drastically improves performance for the same configuration. For example, the winning combination (HardwareEfficient, KL Divergence, SPSA) reaches a fidelity of 0.9847 and a TVD of 0.0747 with 2 layers (rank 1), versus only 0.9321 fidelity and 0.1992 TVD with a single layer (rank 13).

In this noisy environment, the gradient-free SPSA optimizer demonstrates its theoretical robustness by taking first place in the ranking. Nevertheless, the Adam optimizer proves to be an excellent challenger (rank 2 with a fidelity of 0.9826) while being significantly faster. Training with Adam required only 52.89 seconds, versus 86.81 seconds for SPSA, which makes it a very relevant choice for optimizing computation time.

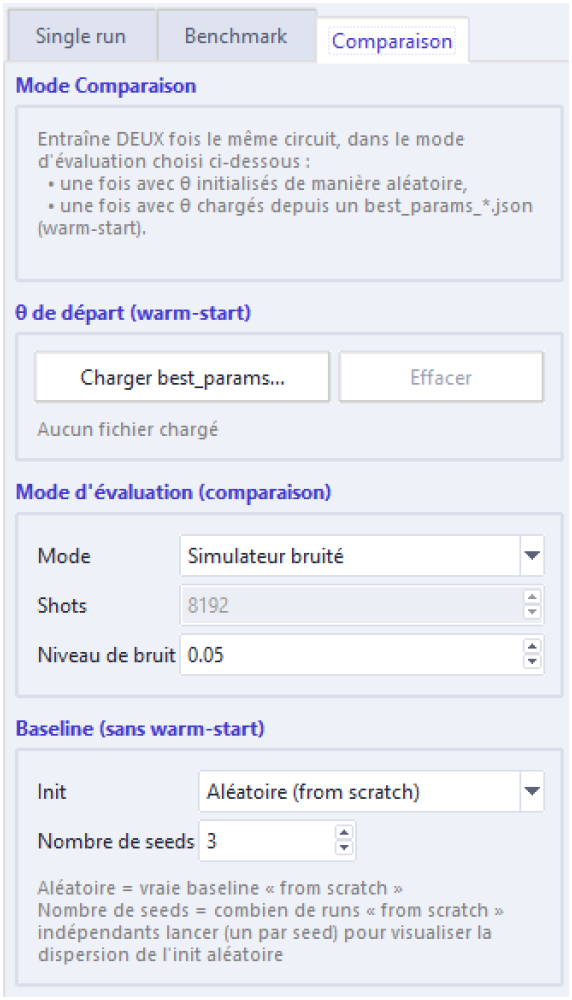
Both the KL Divergence and MMD functions achieve excellent results when paired with a good ansatz. Although the KL Divergence provides the absolute best-performing configuration at rank 1, the MMD proves very stable and occupies ranks 2 and 3 with extremely close metrics.

3.3 The Comparison part (Warm-start)

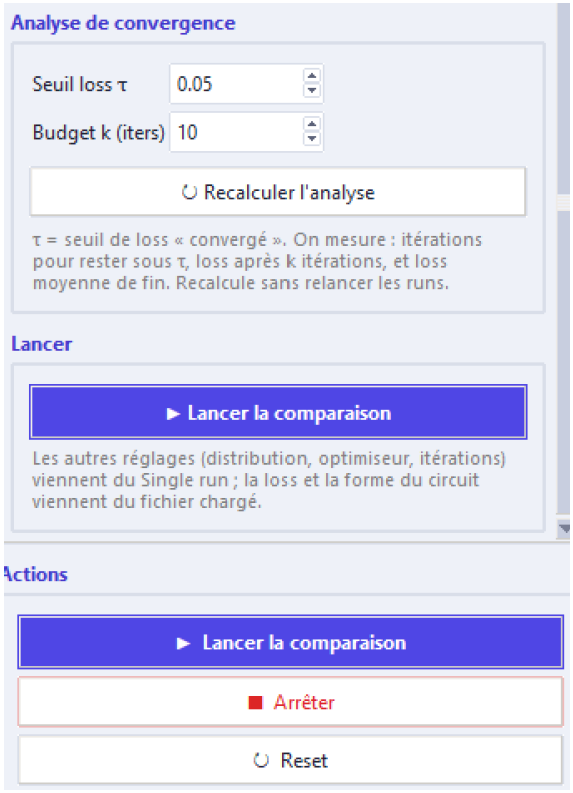
The third tab is dedicated to the comparative study of environments, and specifically of the *warm-start*. The user can correlate two complete configurations.

This feature was essential for us to test the real impact and verify whether exporting the parameters $\boldsymbol{\theta}$ precomputed on a perfect simulator to a QPU execution improved convergence. The interface then offers to link:

- A completely naive *from scratch* run on a noisy target.
- A retrieval of parameters from a previous training run (*Warm-start*).



(a) Selection of the 'From Scratch' configuration



(b) Selection of the transfer configuration

Figure 20: Parameter tabs for initializing the comparative study.

To summarize, QCBM Lab offers an interface that is simple to use, intuitive and oriented towards QML experimentation. A person without a heavy technical background can easily launch their own circuits and instantly compare the architecture across the different metrics. In the long run, additional modules could be connected to it, such as asynchronous transfer of analyses via the cloud.

4 Experimental results

4.1 Analysis of a single run on the Log-normal distribution (1 to 8 qubits)

Parameter	Value
Ansatz type	HardwareEfficient
Number of qubits (n_{qubits})	1-8
Number of layers (n_{layers})	3
Cost function (Loss)	KL divergence
Target distribution	Log-normal
Optimizer	SPSA
Max number of iterations	100
Seed (random seed)	42

Table 8: Experimental configuration of the QCBM model

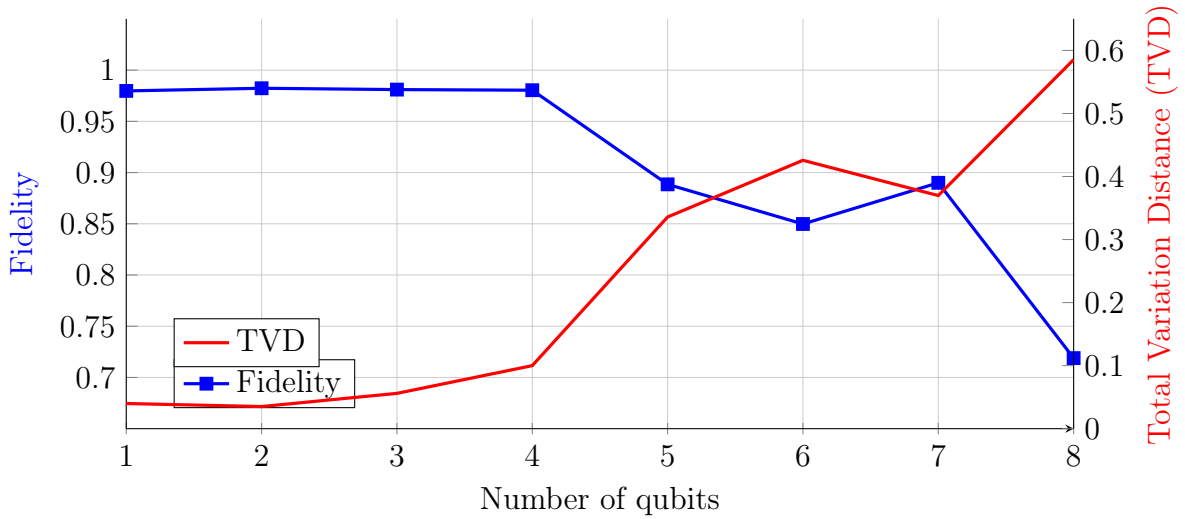


Figure 21: Evolution of the Fidelity and the TVD as a function of the number of qubits.

4.2 Analysis of the results: evolution of QCBM performance

The graph in Figure 21 relates our two metrics to evaluate the quality of the distribution learned by the QCBM as a function of the number of qubits (from 1 to 8).

We observe a clear transition in the behavior of the model, which splits into two distinct phases.

4.2.1 1. High-precision regime (from 1 to 4 qubits)

For configurations ranging from 1 to 4 qubits, the QCBM shows very good performance. The *Fidelity* remains constant at a near-perfect level (around 0.98), while the *TVD* stays at very low values, below 0.10.

In this regime, the Hilbert state space is relatively small (from $2^1 = 2$ to $2^4 = 16$ possible states). A *HardwareEfficient* ansatz with only 3 layers has more than enough expressivity to

capture the full complexity of a Log-normal distribution. The SPSA optimizer thus converges without difficulty to the global minimum.

4.2.2 2. Performance degradation and instability (from 5 to 8 qubits)

From 5 qubits onwards, we observe a striking break in the metrics. The *Fidelity* drops sharply, reaching its lowest point at 0.71 for 8 qubits. At the same time, the *TVD* shoots up, reaching nearly 0.58.

We note, however, a slight anomaly (or performance spike) at 7 qubits, where the *Fidelity* temporarily rises back to 0.89 and the *TVD* drops back to 0.37. This is typical behavior of stochastic optimizers such as SPSA, which can sometimes converge to a better local minimum depending on the random initialization (*seed*).

4.2.3 Theoretical interpretation of the performance drop

This degradation of the results as the system size increases is mainly explained by three factors inherent to variational quantum algorithms:

Dimensionality explosion: With each added qubit, the size of the probability space doubles. At 8 qubits, the model must learn a distribution over $2^8 = 256$ states from a limited number of iterations (100).

Under-parameterization of the ansatz: Keeping a fixed number of 3 layers for a system of up to 8 qubits drastically restricts the expressivity of the circuit. To maintain high fidelity, the circuit depth must be increased proportionally in order to generate enough entanglement. The experimental procedure that led to Figure 21, consisting of a succession of single runs progressively increasing the register size, perfectly illustrates this phenomenon. As soon as degradation appeared at 5 qubits with 3 layers (fidelity dropping to around 0.88), a targeted intervention raising the ansatz to 4 layers immediately restored the fidelity to above 0.92. This check confirms that the bottleneck is indeed related to expressivity and supports the hypothesis of a critical ratio required between the system dimension (the number of qubits) and the circuit depth (the number of layers) to correctly map the target distribution.

Barren Plateau phenomenon: As the number of qubits increases, the landscape of the cost function (here the KL divergence) tends to flatten. Gradients become exponentially small, making optimization with SPSA difficult to achieve in only 100 iterations.

In conclusion, QCBM single runs make it possible to perform this kind of benchmark; the user has great freedom since they can influence all the parameters.

5 Warm-start study: initialization from ideal training

5.1 Warm-start advantage on quantum hardware (QPU)

To demonstrate the effectiveness of the *warm-start* method, we compared its performance with a classical random initialization (*from scratch*). Both approaches were executed on a real quantum processor (QPU `ibm_marrakesh`).

On real quantum hardware, circuit execution is costly in time and subject to noise. It is therefore crucial to reach convergence in a minimum number of steps. For this experiment, we limited the budget to only 5 iterations.

Metric	**Random init**	**Warm-start**	**Advantage**
Iters → loss<0.1 (sustained)	not reached	1	warm-start
Loss @ 3 iters	0.283094	0.003758	$\Delta = -0.2793$
Mean loss (last 5)	0.341204	0.006234	$\Delta = -0.335$
Final loss	0.271418	0.004302	$\Delta = -0.2671$
Fidelity	0.927421	0.998551	$\Delta = +0.07113$
TVD	0.297852	0.041992	$\Delta = -0.2559$
Time (s)	424.58	159.71	$\Delta = -264.9$

Table 9: Random init vs warm-start comparison

Iteration	**Loss — Random init**	**Loss — Warm-start**
1	0.337295	0.011619
2	0.451441	0.009014
3	0.283094	0.003758
4	0.362770	0.002479
5	0.271418	0.004302
Final result	loss=0.2714, fid=0.9274	loss=0.0043, fid=0.9986

Table 10: Convergence over 5 QPU iterations (`ibm_marrakesh`)

The results highlight the overwhelming superiority of the *warm-start* approach:

- **Random initialization:** The model fails to converge within the allotted time. The loss function (*loss*) stagnates at high values and fluctuates (going from 0.337 at the first iteration to 0.271 at the fifth). The final fidelity obtained is 0.9274.
- **Warm-start:** By injecting pre-optimized parameters, the model starts in an extremely favorable region. From the very first iteration, the loss is already minimal (0.0116) and goes down to 0.0043 by the end of the fifth iteration. The final fidelity is near-perfect, with a score of 0.9986.

As the metrics and convergence curves show, the *warm-start* not only crosses the convergence threshold (set at a *loss* of 0.03) from the very first iteration, but also guarantees a stability that random initialization is unable to achieve with such a limited iteration budget. This technique therefore proves indispensable for obtaining high-fidelity results on a real QPU in record time.

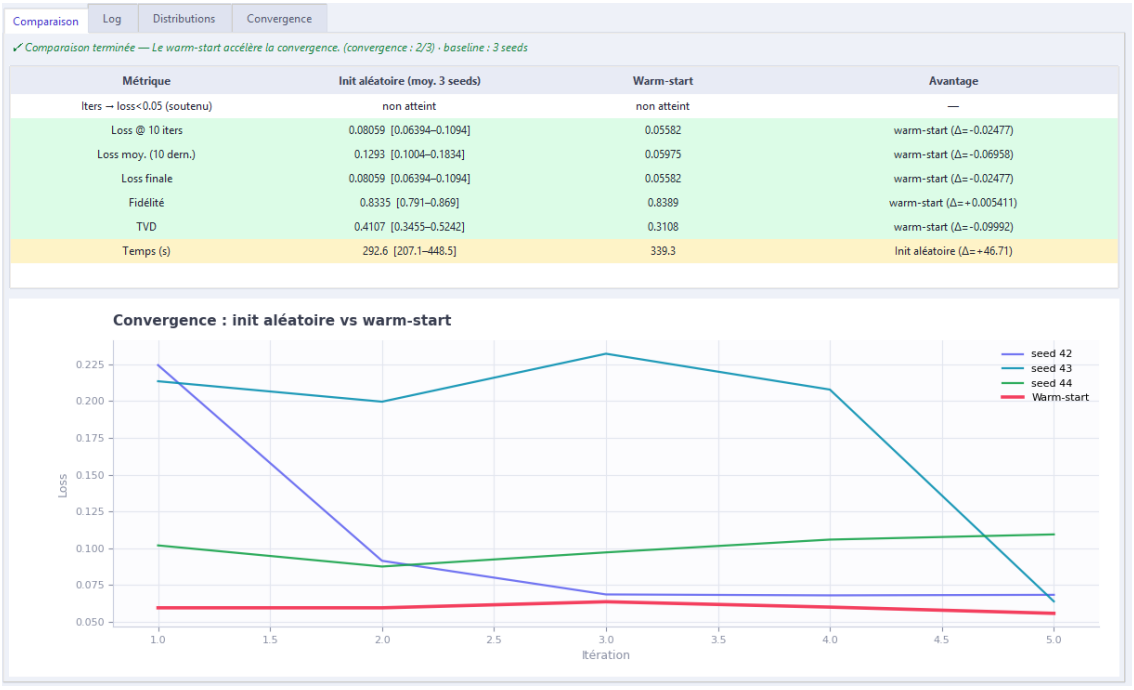


Figure 22: Test on 3 different seeds

The convergence curve shows that the warm-start instantly starts in a very low error region (Loss ≈ 0.06) and remains stable throughout the iterations. Conversely, random initializations start with a much higher Loss.

While seeds 42 and 43 manage, after significant fluctuations, to approach the warm-start level by the 5th iteration, seed 44 perfectly illustrates the risk of a from-scratch initialization: the model stagnates at a Loss above 0.10 without managing to converge. The warm-start eliminates this risk.

On average over the 3 seeds, the qualitative advantage of the warm-start is systematic. The mean final Loss is lowered from 0.08059 to 0.05582, the Fidelity is slightly improved, and above all, the Total Variation Distance (TVD) is significantly reduced (going from 0.4107 to 0.3108).

In short, transferring parameters pre-trained on an ideal simulator to a noisy environment makes it possible to escape the "lottery" of random initializations. The model converges faster, much more predictably, and avoids wasting a precious QPU iteration budget searching for a descent direction in an unfavorable cost landscape.

6 Execution on a real QPU

Beyond the ideal and noisy simulators, QCBM Lab makes it possible to launch training on a **real IBM quantum processor**. This section describes the execution workflow, the automatic hardware selection, the distinction between the *session* and *job* modes, as well as the constraints imposed by the free quota. A complete demonstration closes the chapter.

6.1 Execution workflow

Execution on real hardware is triggered by selecting the **"Real IBM QPU"** evaluation mode. An authentication window then opens and asks for:

- the user's **IBM Quantum API key**;
- the **number of iterations** to run on the QPU (default 5);
- an optional **instance** (e.g. `ibm-q/open/main`).

For security reasons, the key is **never written to disk**: it is kept in memory for the duration of the application session and sent to the backend only for the current run.

Execution then follows a **two-phase hybrid** scheme:

1. **Phase 1 — local training (noisy simulator).** The circuit is first trained on a density-matrix simulator (`default.mixed` with a depolarizing channel), possibly coupled with an Optuna hyperparameter search. Inexpensive, this phase brings the parameters $\boldsymbol{\theta}$ into a favorable region, then *freezes* the hyperparameters.
2. **Phase 2 — refinement on QPU.** A few optimization iterations (typically 5) are run on the real hardware, starting from the pre-trained $\boldsymbol{\theta}$.

The comparison module (warm-start, Chapter 5) can bypass phase 1 (option `skip_local_phase`) in order to optimize directly on the QPU from the initialization, so as to isolate the warm-start advantage on hardware.

6.2 Automatic backend selection

The backend connects to the IBM service via `QiskitRuntimeService`, successively trying the channels `ibm_quantum_platform`, `ibm_quantum` and `ibm_cloud` in order to remain compatible with both recent and older accounts. The QPU is not chosen manually: the call `service.least_busy(operational=True, simulator=False, min_num_qubits=`n`)` automatically selects the **least busy operational real processor** with at least n qubits. The PennyLane device is then built via `qml.device("qiskit.remote", ...)` and connected to this backend.

6.3 "Session" mode versus "job" mode

A QCBM is a **hybrid** algorithm: each iteration chains many circuits — the evaluation of the cost function *and*, for gradient-based optimizers, the parameter-shift estimation (2 circuits per parameter).

Session mode (ideal). QCBM Lab attempts to open a single *Qiskit Runtime Session* (`qiskit_session`) around the entire optimization loop. All circuits are then chained within a *single reserved slot*, without going back through the queue between executions.

Job mode (fallback). If sessions are unavailable — which is the case for IBM's free *Open* plan, which rejects them (error 1352), or if `qiskit_session` is not exposed — the system **automatically** switches to "job" mode: each circuit is sent as an **independent job** and **goes through the IBM queue again** every time. The result remains correct, but the total time explodes, dominated by repeated queue waits.

In practice, on an **Open (free)** account, sessions are not accessible: all our runs were carried out in "job" mode, the main source of slowness observed on real hardware.

6.4 Free quota constraint (10 minutes)

IBM's Open plan provides a budget of approximately **10 minutes of free QPU time per month**. This budget is consumed **very quickly**:

- with a gradient-based optimizer, a single iteration costs $2d + 1$ circuits (d = number of parameters);
- in "job" mode, each circuit additionally suffers the queue latency.

Three levers make it possible to stay within the budget:

1. **limit the QPU iterations** (default 5);
2. **prefer a gradient-free optimizer** (SPSA: ~ 2 circuits per iteration instead of $2d+1$);
3. **use warm-start** (Chapter 5), which reaches high fidelity from the first iteration.

6.5 Demonstration: learning a Gaussian on a QPU

To illustrate the end-to-end workflow, we train a QCBM on a **Gaussian** distribution with **3 qubits**, using the **HardwareEfficient** ansatz, the **KL divergence** and the **SPSA** optimizer — chosen for its low circuit cost, suited to the free quota.

Parameter	**Value**
Target distribution	Gaussian
Number of qubits (n)	3
Ansatz	HardwareEfficient
Number of layers (L)	2
Cost function	KL divergence
Optimizer	SPSA
QPU iterations	10
IBM backend	`ibm_marrakesh` (automatic selection)

Table 11: Configuration of the real QPU execution demonstration.

Execution starts by entering the API key and the number of iterations in the dedicated window (Figure 23): here, 10 iterations are requested on the hardware.

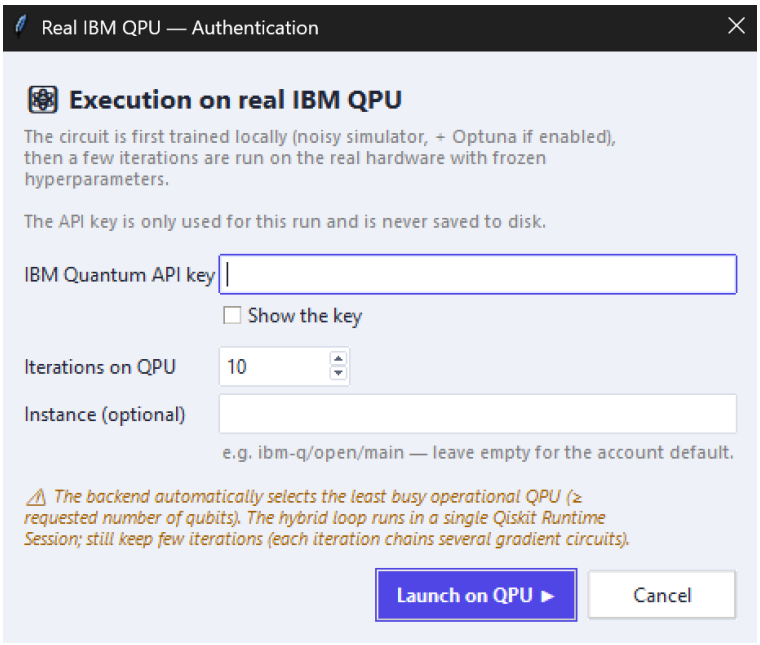


Figure 23: Configuration window for execution on a real QPU.

The progress of the execution can be followed live in the logs (Figure 24), then the distribution learned on the real hardware is compared with the target, along with the evaluation metrics (Figure 25).

```
   [Control] Phase 2/2: connecting to the IBM QPU…
   [Control] Selected QPU: ibm_marrakesh. Running 10 iteration(s) on real hardware (frozen hyperparameters).
   [Control] Qiskit Runtime Session: the hybrid loop (quantum → classical → quantum) runs in a single reserved session.
   [Control] ⚠ Session refused by IBM (RequestsApiError). Open Plan? Falling back to "job" mode: each circuit is sent as an independent job
(slower, but allowed).
   [QPU] Iteration    1 | loss = 0.003302
   [QPU] Iteration    2 | loss = 0.003026
   [QPU] Iteration    3 | loss = 0.002921
   [QPU] Iteration    4 | loss = 0.002634
   [QPU] Iteration    5 | loss = 0.002867
   [QPU] Iteration    6 | loss = 0.002749
   [QPU] Iteration    7 | loss = 0.003071
   [QPU] Iteration    8 | loss = 0.002332
   [QPU] Iteration    9 | loss = 0.002466
   [QPU] Iteration   10 | loss = 0.003447
--------------------------------------------------
✓ Simulation finished in 268.62s
  Executed on the IBM QPU: ibm_marrakesh
```

Figure 24: Real QPU execution logs: the Qiskit Runtime Session is rejected by IBM (Open plan), the system then automatically switches to “job” mode, and the 10 iterations run one after another on `ibm_marrakesh`.



Figure 25: Gaussian distribution learned on a real QPU (red) compared with the target (blue), with the associated metrics (fidelity, TVD, time).

Metric	**Value**
Fidelity	0.9916
TVD	0.0712
QPU iterations	10
Total time	268.62 s
Execution mode	"job" (Session rejected, Open plan)
Backend	`ibm_marrakesh`

Table 12: Results of the real QPU demonstration (`ibm_marrakesh`).

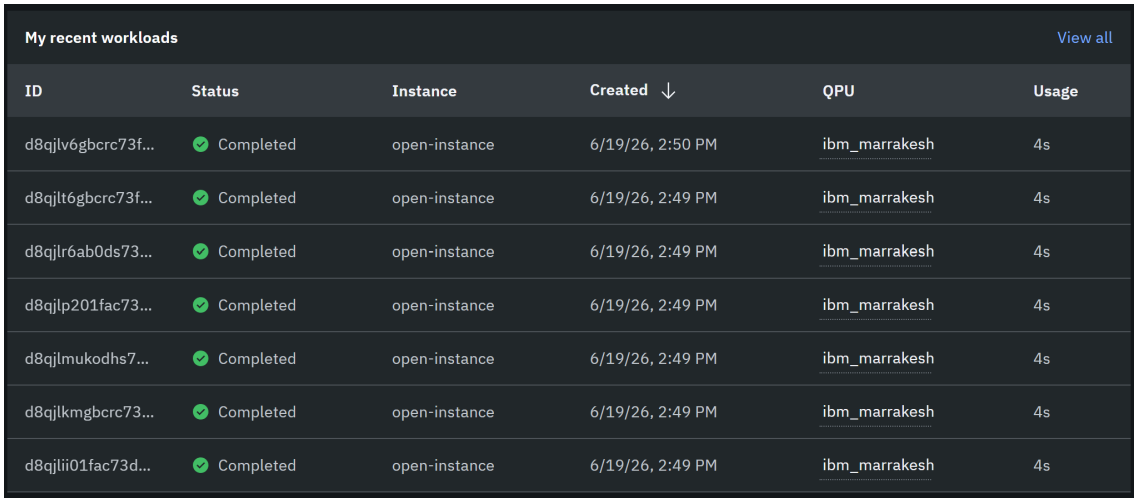
Analysis of the results.

Training on real hardware reaches a **fidelity of** 0.9916 and a **TVD of** 0.0712 in only 10 iterations: the Gaussian distribution is therefore reproduced with high accuracy directly on the QPU. The figure 25 visually confirms this agreement — the bell shape is perfectly reproduced, with the central states $|011\rangle$ and $|100\rangle$ concentrating most of the probability mass. The few residual deviations lie on the low-probability tail states ($|000\rangle$, $|001\rangle$ and $|111\rangle$ slightly overestimated), a typical signature of **hardware noise**: on a NISQ processor, states that should be nearly zero receive a small parasitic mass due to gate and measurement

errors.

The logs (Figure 24) concretely illustrate the *session vs job* issue: the Qiskit Runtime Session is **rejected by IBM** (account on the Open plan), which forces the automatic fallback to "job" mode. Since each circuit then goes back through the queue, the complete run required 268.62 **s** for 10 iterations. The choice of the **SPSA** optimizer ($\sim$ 2 circuits per iteration, versus $2d + 1$ for a gradient-based optimizer) is precisely what keeps this cost within the free quota: a gradient-based configuration would have multiplied the number of jobs — and therefore the waiting time — by a factor proportional to the number of parameters.

This "job" mode operation is confirmed on the IBM Quantum Platform side (Figure 26): the dashboard lists a succession of independent *workloads*, all executed on `ibm_marrakesh`, each corresponding to a circuit submitted separately rather than to a single reserved session.



My recent workloads — View all

ID	Status	Instance	Created ↓	QPU	Usage
d8qjlv6gbcrc73f...	Completed	open-instance	6/19/26, 2:50 PM	ibm_marrakesh	4s
d8qjlt6gbcrc73f...	Completed	open-instance	6/19/26, 2:49 PM	ibm_marrakesh	4s
d8qjlr6ab0ds73...	Completed	open-instance	6/19/26, 2:49 PM	ibm_marrakesh	4s
d8qjlp201fac73...	Completed	open-instance	6/19/26, 2:49 PM	ibm_marrakesh	4s
d8qjlmukodhs7...	Completed	open-instance	6/19/26, 2:49 PM	ibm_marrakesh	4s
d8qjlkmgbcrc73...	Completed	open-instance	6/19/26, 2:49 PM	ibm_marrakesh	4s
d8qjlii01fac73d...	Completed	open-instance	6/19/26, 2:49 PM	ibm_marrakesh	4s

Figure 26: IBM Quantum Platform dashboard: each iteration of the hybrid loop is submitted as a separate *workload* (job) on the QPU `ibm_marrakesh`, confirming execution in "job" mode.

7 Use case: Quantum Monte-Carlo

The hybrid quantum-classical (NISQ) approach makes it possible to improve the scenario generation step of the Monte Carlo method, by replacing classical pseudo-random generators with parameterized quantum circuits (QCBM). In this first case study, we validate the architecture by simulating the behavior of a single asset (Microsoft).

7.1 Creation and encoding of the target distribution (single asset)

Unlike classical models, which often rely on a Gaussian distribution, real markets exhibit frequent extreme events (crashes, rallies). To model these *fat tails* on our reference asset (Microsoft), we define a target distribution based on Student's t-distribution. Specific to the Monte-Carlo module, this distribution differs from the eight benchmark distributions presented in Chapter 2 and is written as:

$$f(x) = \left(1 + \frac{(x-\mu)^2}{\nu\sigma^2}\right)^{-\frac{\nu+1}{2}} \tag{3}$$

Where μ is the mean return (*drift*), σ the volatility, and ν the degrees of freedom. To load this distribution onto a quantum register, the continuous space of returns is discretized into orthogonal states, thus creating a dictionary mapping each quantum state to a specific daily return.

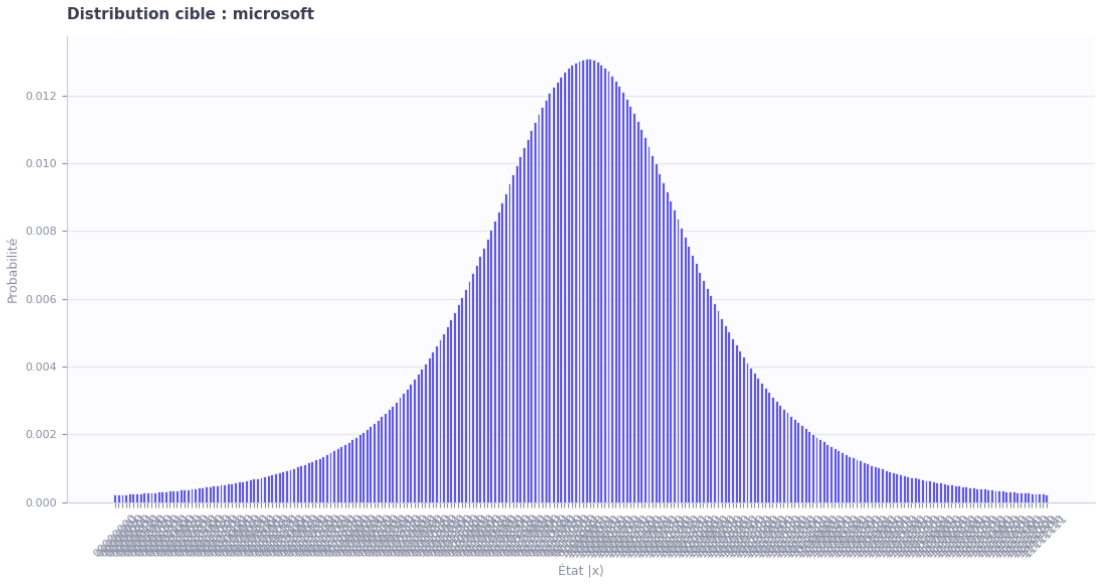


Figure 27: Target distribution (Microsoft model) discretized over the states of the quantum register. The central probability peak and the characteristic fat tails modeling extreme variations can be observed.

7.2 Trajectory simulation and risk assessment

The binary states measured at the output of the circuit are retrieved and translated into daily returns r_t. The evolution of the initial portfolio C_0 (here set to €10,000) over a horizon T of 252 days is then computed classically:

$$C_T = C_0 \prod_{t=1}^{T} (1 + r_t) \tag{4}$$

Repeating this process over thousands of trajectories makes it possible to visualize the range of possible scenarios and to observe the mathematical battle between the *drift* (which pulls the capital upwards) and the *Volatility Drag* (which erodes it due to variance).

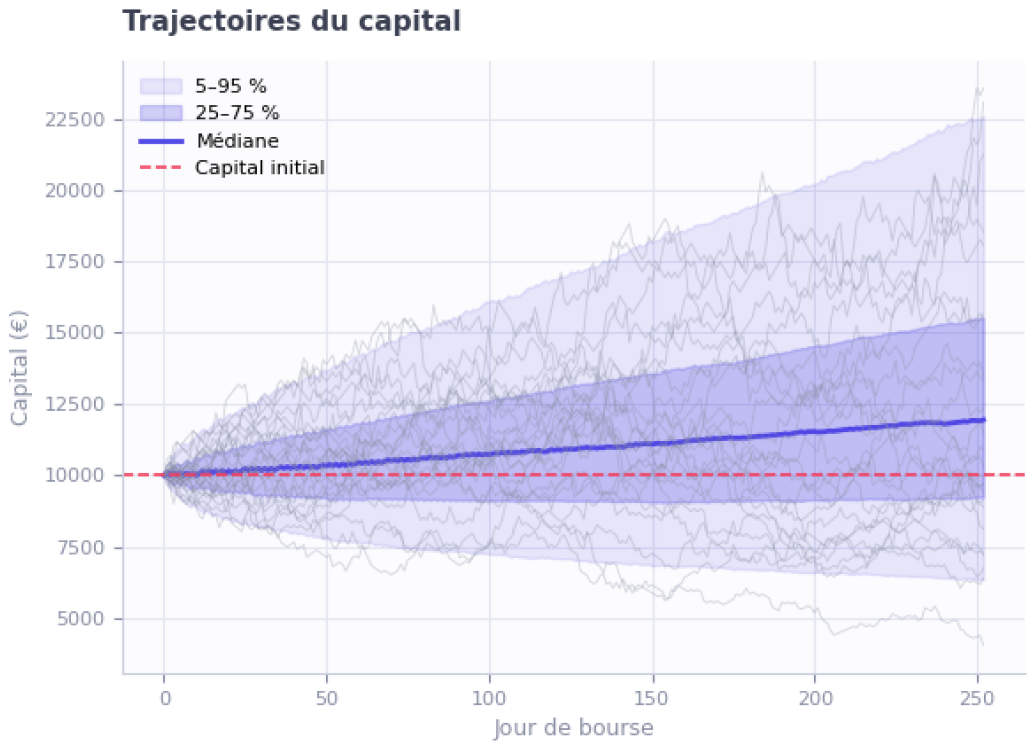


Figure 28: Evolution of the Monte Carlo trajectories over 252 days. The dark blue line represents the median, which here manages to rise above the initial capital (dashed red line) thanks to a positive expected return.

The quantitative analysis of these trajectories makes it possible to extract the key performance and risk indicators (Value at Risk) of the simulated portfolio.

Indicateur	Valeur
Capital initial	10 000.00 €
Capital médian à 252 jours	11 955.55 €
Capital moyen	12 912.87 €
Scénario pessimiste (5e percentile)	6 345.82 €
Scénario optimiste (95e percentile)	22 620.28 €
Pire scénario simulé	3 006.19 €
Meilleur scénario simulé	55 046.94 €
Probabilité de finir en perte	32.1 %

Figure 29: Key indicators of the simulation. For an investment of €10,000, the median scenario stands at nearly €11,956, with a probability of loss contained at 32.1%.

Finally, the cross-section at the 252nd day makes it possible to observe the final distribution of the capital. The marked right skew confirms the presence of compound interest acting on a basis of volatile returns.

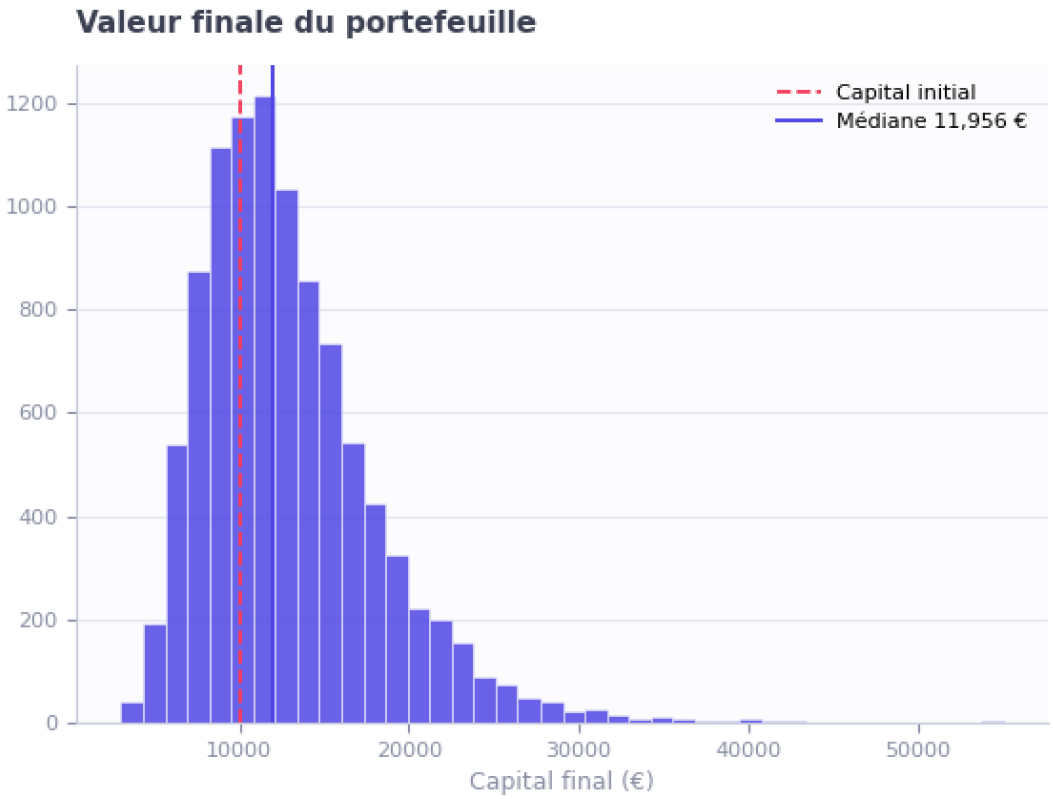


Figure 30: Histogram of the final portfolio value at $T = 252$ days. The distribution takes a log-normal shape, illustrating the impact of extreme bullish scenarios on the expectation relative to the median.

7.3 Perspectives: quantum advantage through multidimensional correlation

Although this PoC validates the simulation architecture on a single asset, the classical computer currently remains more efficient for this type of one-dimensional task. True Quantum Advantage only emerges when moving to multidimensional portfolios (e.g. 50 or 100 linked stocks).

In a classical setting, simulating the dependencies between multiple assets requires massive covariance matrices and very heavy heuristic computations to overcome the curse of dimensionality. Conversely, the quantum approach makes it possible to exploit the **physical entanglement** between qubits. The QCBM then becomes capable of natively capturing the deep, non-linear correlations of an entire market within a single superposed state, offering sampling of unmatched expressivity for detecting complex systemic risks.

8 Discussion, limitations and perspectives

Although the initial performance on simple or low-dimensional distributions proves promising, learning complex or high-dimensional probability distributions constitutes the major limitation of current Quantum Circuit Born Machine (QCBM) architectures. This section summarizes these obstacles and outlines the perspectives for the evolution of the model.

8.1 Structural and physical limitations of QCBMs

The systematic evaluation carried out during this project makes it possible to identify five interconnected bottlenecks that degrade the model's generalization capacity when facing complex probability landscapes.

1. The ansatz trilemma in the face of complexity.

Learning a highly correlated distribution requires increased unitary expressivity, which requires increasing the depth L of the circuit (number of rotation and entanglement layers). This requirement runs into the fundamental ansatz trilemma set out in our state of the art: high expressivity drastically increases sensitivity to hardware noise. The accumulation of gate errors destroys the coherence of the state before measurement, which nullifies the optimizability of the parameters $\boldsymbol{\theta}$.

2. Dimensionality explosion and under-parameterization.

When the distribution involves a large number of variables, the dimension of the underlying Hilbert space grows exponentially in $O(2^n)$. If the ansatz architecture remains fixed (for example, $L = 3$ fixed layers for $n = 8$ qubits), the PQC suffers from critical under-parameterization. The number of adjustable parameters becomes insufficient to map the fine details of the target distribution, leading to a sharp drop in fidelity and an explosion of the total variation distance (TVD).

3. The mathematical obstacle of barren plateaus.

For distributions with a large number of qubits, the cost function landscape suffers from the *barren plateau* phenomenon. The gradient variance vanishes exponentially according to:

$$\mathrm{Var}\left[\frac{\partial \mathcal{L}}{\partial \theta_i}\right] \in O\left(\frac{1}{2^n}\right).$$

In a complex space, the gradient signal is smaller than the statistical fluctuations of the measurement noise. The optimizers (SPSA, Adam) become unable to identify a descent direction, and the parameter update drifts into a fruitless random walk.

4. State collapse due to induced hardware noise.

In the current NISQ regime, the gate error rate (0.1 to 1 %) and decoherence times restrict the viability of long circuits. When a complex distribution requires a high degree of entanglement (as in the *Bars and Stripes* benchmark), the required depth drives the quantum state towards the maximally mixed state. This phenomenon artificially flattens the optimization landscape (*noise-induced barren plateau*), making the true loss topology undetectable for the processor.

5. Computational cost of shot-based estimation.

Capturing the subtle features of a complex distribution — such as the heavy tails of the log-normal law or of the Student's t-distribution used in our Monte-Carlo module — requires very fine statistical convergence. Since the QPU only provides discrete projections $x \in \{0,1\}^n$, accurate gradient estimation with an acceptable tolerance ε requires a volume of measurements exponential in $O(2^n)$, which proves prohibitive under real operating conditions.

8.2 Perspectives and avenues for improvement

To overcome these limitations and tackle large-scale industrial use cases, several development directions should be considered:

- **Structured and physics-inspired ansatzes:** Move away from generic *Hardware-Efficient* (HEA) structures in favor of ansatzes guided by the structure of the data or by the physics of the problem (such as MPS or PEPS tensor networks), naturally limiting the proliferation of redundant parameters.
- **Local cost functions:** Favor local variants of information-based losses or kernel distances to delay the appearance of global barren plateaus, preserving usable gradients in high dimension.
- **Error mitigation:** Systematically integrate *Zero-Noise Extrapolation* (ZNE) or state purification protocols to extract the clean probabilistic signal from measurements corrupted by thermal noise.
- **Generalization of warm-start:** Exploit the potential of transferring parameters pre-optimized on classical architectures (ideal or weakly noisy) in order to stabilize and constrain the search space during the final calibration phases on physical processors.
- **Custom ansatz editor:** Allow users to design and integrate their own variational architectures directly from the graphical interface. Such a feature would turn QCBM Lab into a true research platform, capable of quickly testing new circuit topologies, evaluating architectures from recent literature or comparing different entanglement strategies without modifying the source code.
- **Automatic analysis of expressivity and barren plateaus:** Add an analysis module capable of estimating, before training, indicators such as the expressivity of the ansatz, its entangling capability, its effective depth or the risk of barren plateaus. These diagnostics would help guide the choice of architectures and avoid costly benchmark campaigns on poorly optimizable circuits.
- **Automated quantum architecture search:** Extend the use of Optuna beyond hyperparameter optimization to automatically explore the space of circuit architectures. Such an approach could dynamically select the number of layers, the rotation gates, the entanglement schemes or the topologies best suited to a given target distribution, paving the way for assisted ansatz design for QCBMs.

9 Conclusion

This project addressed the learning of probability distributions using Quantum Circuit Born Machines, from theory to execution on real quantum hardware, relying on the interactive tool QCBM Lab. The experiments conducted provide an answer to the three questions raised in the introduction.

Ansatz / loss / optimizer combination.

The benchmark comparative study showed that an ansatz with entanglement — first and foremost HardwareEfficient — clearly outperforms architectures without entanglement (RotationOnly) on correlated distributions such as the Log-normal. The KL divergence and the MMD proved to be the most reliable cost functions, while Adam (on an ideal simulator) and SPSA (in the presence of noise) are the most effective optimizers. The right setting therefore depends on the execution regime: gradient-based optimizer on simulator, gradient-free on QPU.

Noise robustness.

At fixed depth, increasing the number of qubits strongly degrades the fidelity (from $\sim$0.98 at 4 qubits down to $\sim$0.72 at 8 qubits), under the combined effect of under-parameterization, barren plateaus and hardware noise. The levers that improve robustness are short circuits, gradient-free optimizers (SPSA) and maintaining a sufficient depth/qubit ratio.

Contribution of the warm-start.

Transferring parameters pre-optimized on an ideal simulator to a noisy environment — including on the real QPU `ibm_marrakesh` — proved decisive: convergence from the first iteration, near-perfect fidelity (0.9986) and execution time cut in half, whereas random initialization fails to converge within the same budget. The warm-start removes the "lottery" of initialization and constitutes the most promising lever for exploiting current NISQ hardware.

Ultimately, although the QCBM does not yet outperform classical methods on low-dimensional tasks, this project validates a complete pipeline — from design to execution on real hardware — and highlights where its true value lies: capturing complex correlations in high dimension, where quantum entanglement could offer a decisive advantage.

Source code. The full code of the project and of the QCBM Lab tool is publicly available on GitHub:
`https://github.com/VasicIvann/Quantum_Distribution_Simulation`

References

[1] M. Cerezo *et al.*, *Variational quantum algorithms*, Nature Reviews Physics, 2021.

[2] M. Benedetti, E. Lloyd, S. Sack, M. Fiorentini, *Parameterized quantum circuits as machine learning models*, Quantum Science and Technology, 2019.

[3] J. R. McClean *et al.*, *Barren plateaus in quantum neural network training landscapes*, Nature Communications, 2018.

[4] M. Schuld, N. Killoran, *Quantum machine learning in feature Hilbert spaces*, Physical Review Letters, 2019.

[5] A. Pérez-Salinas, A. Cervera-Lierta, E. Gil-Fuster, J. I. Latorre, *Data re-uploading for a universal quantum classifier*, Quantum **4**, 226, 2020. `https://doi.org/10.22331/q-2020-02-06-226`

[6] S. Woerner, D. Egger, *Quantum risk analysis*, npj Quantum Information, 2019.

[7] PennyLane QCBM tutorial, `https://pennylane.ai/qml/demos/tutorial_qcbm`

[8] T. Akiba, S. Sano, T. Yanase, T. Ohta, M. Koyama, *Optuna: A Next-generation Hyperparameter Optimization Framework*, Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD), 2019. `https://doi.org/10.1145/3292500.3330701`

[9] Y. Chen, *A Hybrid Quantum-Classical Framework for Financial Volatility Forecasting Based on Quantum Circuit Born Machines*, arXiv preprint arXiv:2603.09789, 2026. `https://arxiv.org/abs/2603.09789`

[10] Y. Gujju, R. Harang, T. Shibuya, *LLM-Guided Ansätze Design for Quantum Circuit Born Machines in Financial Generative Modeling*, arXiv preprint arXiv:2509.08385, 2025. `https://arxiv.org/abs/2509.08385`